Summary of Book4 Sans 401

Cryptosystem Fundamentals
Cryptography, the science of hidden writing, helps us communicate without revealing the meaning of
information to adversaries. It also potentially validates to whom we are communicating. It can protect any kind
of data, from very sensitive information, such as internet-based commerce and banking transactions, to
harmless messages you would prefer that no one else knew about, such as a letter to a friend. Cryptography
(abbreviated as crypto) can provide a great deal of confidentiality and integrity checks for information.
However, it is not a silver bullet, and it can lead to a tremendous false sense of security unless used properly
and implemented correctly. Cryptography should always be a part of a larger defense-in-depth strategy,
providing just one layer of the security onion

Cryptography – Simple Explanation

No cryptographic method is 100% secure

We assume an algorithm is not secure until it has been tested and attacked many times (cryptanalysis).

Even then, it is only assumed secure, not guaranteed forever.

What is secure today might be broken in the future.

Why This Matters

Security is based on testing and proof, not trust.

Over-confidence in cryptography is dangerous.

This idea applies to all information security, not just encryption.

Cryptography vs Cryptanalysis

Cryptography: Creating secure methods to protect data

Cryptanalysis: Trying to break those methods

Together, they are part of cryptology

What Is an Algorithm?

An algorithm is a step-by-step set of instructions

In encryption:

The computer takes a normal (plain) message

Applies mathematical steps

Produces an encrypted (protected) message

Algorithm vs Cipher (Important!)

Encryption algorithm: The actual mathematical process

Cipher: A general term meaning a message written in secret

People often say “encryption cipher”

This usually means encryption algorithm

The wording is not perfect, but it’s commonly used

Why Correct Terms Matter

Using the wrong terms can cause misunderstanding

Misunderstanding leads to bad implementation

Bad implementation leads to high security risk

Security systems fail not because crypto is weak, but because it is used incorrectly

What is Cryptography?

Cryptography means “hidden writing.”

Its goal is to protect information from people who are not allowed to see it.

This is important because someone could be listening when data is sent over a network.

Plaintext

Plaintext is the original message

It is readable and makes sense

Example:

Hello Fares

Plaintext is not only text

Files, programs, images, and executables are also plaintext if they are not encrypted

Encryption

Encryption is the process of locking the message

It uses a mathematical algorithm

The goal is to hide the meaning of the message

Ciphertext

Ciphertext is the encrypted message

It looks like random data and cannot be understood

Example:

X9#Aq!2L@

Decryption

Decryption is the process of unlocking the message

It turns ciphertext back into plaintext

Only someone with the correct key can do this

Simple Flow (Easy to Remember)
Plaintext
↓ (Encryption)
Ciphertext
↓ (Decryption)
Plaintext

Why This Matters

Even if an attacker captures the data, they cannot understand it

Cryptography protects:

Passwords

Emails

Network traffic

Files and databases

What is a Cryptosystem?

A cryptosystem is the complete system used to protect information.

It includes:

The message

The algorithm

The key

The encrypted result

And people who use it

Core Parts of a Cryptosystem
1. All Possible Plaintexts

This means any data before encryption

Examples:

Text: "I went to the store"

Files: documents, images, MP3s

Programs or system files

Anything unencrypted is plaintext

2. Algorithm

The rules or math used to encrypt and decrypt

Example algorithms:

AES

RSA

The algorithm does not change, but the key does

3. Keys

A key is the secret value used by the algorithm

Without the correct key:

Encrypted data cannot be decrypted

If the key is stolen, security is broken

4. All Possible Ciphertexts

These are all encrypted outputs

Ciphertext:

Looks random

Has no meaning without the key

Every plaintext + key = a ciphertext

Humans Are the Weakest Part

Algorithms are usually very strong

People manage the keys

Attackers often:

Trick users (phishing)

Steal passwords or keys

If a user loses or reveals the key, the cryptosystem fails

Attackers target humans, not math

Simple Diagram (Easy to Remember)
Plaintext + Key
↓
Algorithm
↓
Ciphertext

Keys (Simple Explanation)
What is a Key?

A key is a secret value used by a cryptographic algorithm

The algorithm can be public

The key must stay secret

Security depends on protecting the key, not hiding the algorithm

Why Keys Are Important

Keys unlock encrypted data

Just like:

House keys unlock houses

Car keys unlock cars

If someone gets the key → security is broken

Keys Allow Public Algorithms

Cryptographic algorithms are:

Public

Well-tested

Anyone can know how the algorithm works

Only the key is secret

✅ This is good security design

Keyspace (Very Important Concept)

What is Keyspace?

Keyspace = all possible keys

Example:

A 2-digit PIN → 100 possible keys

A 4-digit PIN → 10,000 possible keys

Bigger keyspace = harder to guess

Brute Force Attack

Attacker tries every possible key

Works easily if:

Keyspace is small

Fails if:

Keyspace is huge

Key Length = Strength
Example:

40-bit key

2^{40} possible keys

Can be broken with modern computers

128-bit key

2^{128} possible keys

Practically impossible to brute force

The increase in strength is exponential, not just “a bit stronger”

Real-World Analogy (Easy to Remember)

Luggage locks

Few keys

Many bags open with the same key

❌ Small keyspace

Car keys

Much more unique

Hard to guess

✅ Larger keyspace

Crypto keys

Millions of times larger than car keys

Must be impossible to guess

Keys Must Be Protected… and Yet Usable (Simple)

1. Longer Keys = Harder to Break

Longer keys mean more possible combinations

This makes brute-force attacks much harder

Example:

Short key → easy to guess ❌

Long key → almost impossible to guess ✅

2. Key Length Alone Is NOT Enough

Even the strongest key is useless if:

Someone steals it

Someone forces a user to reveal it

If the attacker has the key → encryption is defeated

Protection of the key is just as important as key length

3. The Key Must Be Usable

Encryption and decryption require access to the key

A system cannot encrypt or decrypt without the key

So we have a problem:

If the computer can access the key, why can’t an attacker?

4. Why Attackers Can’t Easily Access Keys

Computers do not remember keys like humans

Computers:

Read keys from secure storage

Use them only when needed

Attackers must:

Break into the system

Bypass protections

Steal the key from storage or memory

5. The Core Challenge (Very Important)

Keys must be:

Protected from attackers

Accessible to authorized systems

This balance is called Key Management

6. Key Management (Big Picture)

Key management includes:

Where keys are stored

Who can access them

How keys are rotated

How keys are backed up

How compromised keys are revoked

Bad key management = broken security, even with strong crypto

Key Takeaways (Easy to Remember)

Longer keys help against brute force

Stolen keys break encryption instantly

Computers read keys from storage

Keys must be secure but usable

Key management is critical

. Main Cryptography Categories
Encryption – Hide Data

Used to keep data secret

A) Symmetric Encryption (Fast)

Same key encrypts and decrypts

Very fast → used for large data

Examples:

AES (Rijndael) ✅ (most used today)

DES, 3DES ❌ (old)

Blowfish, RC4 (mostly deprecated)

Used for:

Disk encryption

VPN traffic

Secure network communication

B) Asymmetric Encryption (Slow)

Uses two keys:

Public key

Private key

Slower but more flexible

Examples:

RSA

ECC

ElGamal, LUC

Used for:

Secure key exchange

Digital signatures

Certificates

2. Stream vs Block Ciphers

Stream Ciphers

Encrypt data bit by bit

Fast, but risky if misused

Examples:

RC4 ✕

SEAL, WAKE

Block Ciphers

Encrypt data in fixed-size blocks

More secure and common

Examples:

AES ✅

DES, 3DES

AES (block cipher) is the enterprise standard

3. Digital Signatures – Prove Identity

Used to prove:

Who sent the message

Message was not changed

Uses:

Asymmetric crypto

Hashing

Examples:

RSA

DSA

ECC

Used in:

Software updates

Emails

Certificates

4. Hashing – Integrity (Not Encryption)

Hashing:

One-way function

You cannot reverse it

Used to detect changes

Examples:

MD5 ❌

SHA-1 ❌

SHA-2 / SHA-3 ✅

Used for:

Password storage

Digital signatures

File integrity

5. Key Exchange – Share Keys Securely

Used to safely share symmetric keys

Examples:

Diffie-Hellman

IKE (VPNs)

ISA/KMP

SKIP

This solves the problem:

“How do we share a secret key over the internet?”

6. PKI – Public Key Infrastructure

PKI manages:

Certificates

Trust

Identity

Uses:

RSA or ECC

Digital signatures

Hashing

Used in:

HTTPS

VPNs

Email security

Goals of Cryptography (Super Simple)

Cryptography has 4 main goals.
Think of them as security promises

1 Confidentiality – Keep it Secret

Question it answers:
Can anyone else read the message?

Goal:
Only the intended receiver can read the data.

How it's done:
Symmetric encryption (same key)

Examples:

AES ✅ (most common)

DES / 3DES ❌ (old)

RC4 ❌ (weak)

Real-life example:
A locked envelope
Only the person with the key can open it.

2Authentication – Prove Who You Are

Question it answers:
Who sent this message?

Goal:
Verify the identity of the sender.

How it's done:
Asymmetric cryptography
Based on private keys

Examples:

RSA

ECC

ElGamal

Real-life example:
Showing your ID card

3Integrity – Has the Message Changed?

Question it answers:
Was the message modified?

Goal:
Make sure the message is exactly the same as when it was sent.

How it's done:
Hashing

Examples:

SHA (SHA-2 / SHA-3) ✅

MD5 ❌ (broken)

HMAC (hash + secret key)

Real-life example:
A sealed package
If it’s broken, you know something changed.

4 Non-Repudiation – No Denying Later

Question it answers:
Can the sender deny sending it?

Goal:
Prevent someone from saying:
❌ “That wasn’t me”

How it’s done:
Digital Signatures
(Asymmetric crypto + Hash)

Examples:

RSA + SHA

ECC + SHA

Real-life example:
Signing a legal contract
You can’t deny your signature in court.

Quick Summary Table

Goal	What It Means	Crypto Used
Confidentiality	Keep data secret	Symmetric (AES)
Authentication	Prove identity	Asymmetric (RSA/ECC)
Integrity	Detect changes	Hashing (SHA)
Non-repudiation	No denial	Digital signatures

Easy Memory Trick

C A I N

Confidentiality

Authentication

Integrity

Non-repudiation

Big Reality Check

Even the best cryptography can fail if:

Users lose keys

Keys are shared

Weak passwords are allowed

People fall for phishing

Cryptography protects data — people can still break security

General Symmetric Encryption Techniques
Main Goal

The goal of symmetric encryption is to scramble the original message (plaintext) so its meaning is hidden from anyone who does not have the secret key.

The same key is used for:

Encryption

Decryption

Core Techniques Used in Symmetric Encryption
1Substitution

What it does:
Replaces one value with another.

Simple idea:
Each letter/bit is swapped with a different one based on a rule.

Example:

A → D

B → E

Caesar Cipher (classic example)

Key point:
The data changes value, but position stays the same.

2 XOR (Exclusive OR)

What it does:
Uses a mathematical operation between data and a key.

Rules of XOR:

$0 \oplus 0 = 0$

$1 \oplus 0 = 1$

$1 \oplus 1 = 0$

$0 \oplus 1 = 1$

Why XOR is powerful:

Easy to compute

Reversible using the same key

Very important property:

Plaintext $\oplus$ Key = Ciphertext
Ciphertext $\oplus$ Key = Plaintext

Used heavily in:

Stream ciphers

Block cipher internals

3Rotation (Shifting)

What it does:
Moves bits or characters left or right.

Examples:

Bit rotation left

Bit rotation right

Letter shifting (A → C)

Purpose:
Spreads changes across the data (confusion).

Advanced / Combined Techniques
4 Permutation

What it does:
Rearranges the positions of bits or bytes.

Key point:

Values stay the same

Positions change

Example:

Before: ABCD
After: CADB

Purpose:
Makes patterns harder to detect.

5 Hybrid Techniques

What it does:
Combines substitution + permutation.

Why it's used:
Modern encryption needs strong confusion and diffusion.

Examples of algorithms using hybrids:

AES

DES

3DES

Blowfish

Big Picture Summary

Technique	Changes Value?	Changes Position?
Substitution	✅ Yes	❌ No
XOR	✅ Yes	❌ No
Rotation	✅ Yes	❌ No
Permutation	❌ No	✅ Yes
Hybrid	✅ Yes	✅ Yes

Why These Techniques Matter

Alone → weak

Combined → strong encryption

All modern symmetric ciphers use multiple rounds of these techniques

General Symmetric Encryption Techniques

Main Goal

The goal of symmetric encryption is to scramble the original message (plaintext) so its meaning is hidden from anyone who does not have the secret key.

The same key is used for:

Encryption

Decryption

Core Techniques Used in Symmetric Encryption

1Substitution

What it does:
Replaces one value with another.

Simple idea:
Each letter/bit is swapped with a different one based on a rule.

Example:

A → D

B → E

Caesar Cipher (classic example)

Key point:
The data changes value, but position stays the same.

2 XOR (Exclusive OR)

What it does:
Uses a mathematical operation between data and a key.

Rules of XOR:

$0 \oplus 0 = 0$

$1 \oplus 0 = 1$

$1 \oplus 1 = 0$

$0 \oplus 1 = 1$

Why XOR is powerful:

Easy to compute

Reversible using the same key

Very important property:

Plaintext $\oplus$ Key = Ciphertext
Ciphertext $\oplus$ Key = Plaintext

Used heavily in:

Stream ciphers

Block cipher internals

3 Rotation (Shifting)

What it does:

Moves bits or characters left or right.

Examples:

Bit rotation left

Bit rotation right

Letter shifting (A → C)

Purpose:
Spreads changes across the data (confusion).

Advanced / Combined Techniques
4 Permutation

What it does:
Rearranges the positions of bits or bytes.

Key point:

Values stay the same

Positions change

Example:

Before: ABCD
After: CADB

Purpose:
Makes patterns harder to detect.

5 Hybrid Techniques

What it does:
Combines substitution + permutation.

Why it's used:
Modern encryption needs strong confusion and diffusion.

Examples of algorithms using hybrids:

AES

DES

3DES

Blowfish

Frequency Analysis (Simple Explanation)
What is Frequency Analysis?

Frequency analysis is a way to break weak encryption by studying how often letters appear in a message.

Why It Works **!**

Some encryption methods use one-to-one substitution:

Each letter is always replaced with the same letter

Example:

A → Q

B → W

C → E

This is called a monolithic cipher or simple substitution cipher.

The Problem with Simple Substitution

Languages have patterns.

In English:

E appears most often (~13%)

T appears next (~9%)

Common pairs: TH, HE, IN

Common triples: THE

Encryption does not change how often letters appear, only which symbol is used.

How Frequency Analysis Breaks the Cipher

Count how often each encrypted letter appears

Find the most common encrypted letter

Assume it represents E

Next most common → T

Check letter pairs (TH, HE)

Slowly reconstruct the message

Simple Example

Encrypted text:

X Q X X Y X Q

X appears most → likely E

Q next → likely T

Now patterns start to appear.

Why Modern Encryption Is Safe

Modern ciphers:

Use XOR, permutations, rotations

Change data every round

Use keys

Do NOT preserve letter frequency

Frequency analysis does NOT work on AES or modern symmetric ciphers

Permutation (Transposition) – Simple Explanation
What is Permutation?

Permutation means changing the order of letters, not changing the letters themselves.

The same letters stay in the message

Only their positions are shuffled

Key Idea

Substitution changes what the letter is
Permutation changes where the letter is

Simple Example
Plaintext:
ORDER

Positions:

1 2 3 4 5
O R D E R

Key (new order):
3 5 2 1 4

Rearranged letters:
D R R O E

Ciphertext:
DRROE

Important Points

Letters are not hidden, only reordered

Frequency of letters stays the same

An attacker can try all possible orders using a computer

Why Permutation Is Weak ✕

Small number of possible arrangements

Easy to brute-force

Letter patterns still exist

Very easy to break by modern computers

Substitution + Permutation

Modern encryption uses:

Substitution (change the letters)

Permutation (change the order)

Repeated many rounds

Large keys

This makes modern ciphers secure.

Types of Cryptosystems
Three general types of cryptosystems:
Symmetric
• Secret key
• Single or 1-key encryption
Asymmetric
plaintext
plaintext
ciphertext
plaintext
• Public key
ciphertext
plaintext
• Dual or 2-key encryption
Hash
• One-way transformation
plaintext
Hash function

hash
SEC401 | Security Essentials: Network, Endpoint, and Cloud
Types of Cryptosystems
Today, there are three general types of cryptosystems: Secret key or symmetric, public key or asymmetric, and
hash (hashing). Each is used because it provides a different function from the other cryptosystems. These
schemes are usually distinguished from one another by the number of keys employed. The remainder of this
module discusses these different types of systems.

Secure Channel for a Secret Key

When Alice wants to send Bob a secret symmetric key, she can use asymmetric cryptography (public/private keys) to do it safely.

Steps:

Bob shares his public key with Alice.

Public key = can be shared openly

Private key = kept secret by Bob

Alice encrypts the symmetric key using Bob's public key.

Encrypted symmetric key (ciphertext) is sent to Bob.

Bob decrypts the ciphertext using his private key.

Now Bob has the original symmetric key

Why Use This Method?

Asymmetric encryption is slow for large messages

Symmetric encryption is much faster for big messages

So, we combine both:

Hybrid approach:

Asymmetric encryption → secure the symmetric key

Symmetric encryption → fast encryption of actual messages

Example:

Symmetric key size: 128 bits

Document size: 167 KB (~1.3 million bits)

Encrypting the small key asymmetrically is much faster than encrypting the entire document.

Diffie-Hellman Key Exchange

Diffie-Hellman allows two parties to generate the same secret key over an insecure network, even if an eavesdropper is listening.

It's widely used for secure key exchange without transmitting the key itself.

Steps:

Agree on public parameters:

A large prime number,

N

A generator,

G (where

<

G<N)

Choose private keys (kept secret):

Alice:

X
A

Bob:

X
B

Compute public keys (shared openly):

=

m
o
d

Y
A

=G
X
A

modN

=

m
o

d

Y
B

=G
X
B

modN

Exchange public keys.

Compute shared secret key independently:

=

m
o
d

K
A

=Y
B
X
A

modN

=

m
o
d

K
B

=Y
A
X
B

modN

Both parties arrive at the same K because

=

.

m
o
d

K=G
X
A

. X

B

modN
Example from Your Notes:

Public values:

=
7
N=7,

=
4
G=4

Private keys:

=
2
X
A

=2,

=
3
X
B

=3

Compute public keys:

Alice:

=
4
2
m
o
d

7
=
16
m
o
d

7
=
2
Y
A

=4
2
mod7=16mod7=2

Bob:

=
4
3
m
o
d

7
=
64
m
o
d

7
=
1
Y
B

=4
3
mod7=64mod7=1

Exchange public keys:

Alice gets

=
1
Y
B

=1

Bob gets

=
2
Y
A

=2

Compute shared secret key:

Alice:

=
1
2
m
o
d

7
=
1
K
A

=1
2
mod7=1

Bob:

=
2
3
m
o
d

7
=
8
m
o
d

7
=
1
K
B

=2
3
mod7=8mod7=1

✅ Shared secret key: K = 1

Hash Functions
Definition

A hash function is a one-way, irreversible transformation of data that does not use a key. It converts arbitrary-length input (plaintext) into a fixed-length output, called a hash value or message digest.

Irreversible: You cannot recover the original plaintext from the hash.

Fixed-length: The hash output is always the same length, regardless of the input size.

Primary use: Ensures message integrity—detecting any change in data.

How It Works

Start with a message

M of any length.

Apply a hash function (e.g., SHA-256) to generate a hash value
h
h.

Resulting hash is a digital fingerprint of the message.

Example:

Message: "Message"
SHA-256 Hash:

c7cfaff189585980c90c54be0b56980ebc84badd0b95d35dad037353e61a9933

Even a small change in the message produces a completely different hash.

Key Characteristics

No key is used (unlike symmetric or asymmetric encryption).

One-way: Cannot reverse the process to get the original message.

Collision resistance: Extremely low probability that two different messages produce the same hash. If it happens, it’s called a collision.

Fixed-length output: For example, SHA-256 always produces a 256-bit hash.

Common Hash Algorithms

Algorithm	Hash Length	Notes
MD2	128-bit	Byte-oriented, designed for smart cards
MD4	128-bit	Fast software processing
MD5	128-bit	Improved over MD4, slower, known weaknesses
SHA-1	160-bit	Legacy use, vulnerable to collisions
SHA-2	224/256/384/512-bit	Secure, widely used
SHA-3	Variable	Latest standard, highly secure
RIPEMD-160	160-bit	Alternative to SHA-1

Note: Weaker algorithms like MD4, MD5, and SHA-1 should be avoided when possible due to known vulnerabilities.

Use Case in Cryptography

Message integrity: Verify that a message has not been altered.

Digital signatures: Hash is often signed rather than the entire message.

Password storage: Store hash of passwords, not the plaintext.

Hashing is a critical component of cryptography and is considered a third form of cryptographic algorithm alongside symmetric and asymmetric encryption.

Collisions in Hashing

Definition

A hash collision occurs when two different inputs produce the same hash output.

Hash functions produce fixed-length outputs, so if the number of possible inputs exceeds the number of possible outputs, collisions are inevitable (pigeonhole principle).

Collisions are critical because the main goal of hashing is to produce a unique fingerprint for each input.

Why Collisions Matter

Hashing is used for message integrity, digital signatures, and verification.

If two different files produce the same hash:

You cannot reliably detect tampering or verify authenticity.

Digital signatures based on hashes may fail, since the same hash could represent multiple different files.

Reducing the Risk of Collisions

Secure, modern hash algorithms are designed to minimize collisions by ensuring:

Avalanche effect: Small changes in input produce drastically different outputs.

Unpredictability: It is computationally infeasible to predict or force a collision.

Large bit length: More bits allow more unique outputs, reducing the chance of collisions.

Example:

A 128-bit hash (like MD5) has a smaller output space → higher chance of collision.

A 256-bit hash (like SHA-256) has a much larger output space → far less likely to collide.

Key Takeaways

Collisions are a natural consequence of hashing due to fixed output length.

Choosing strong, modern hash functions (SHA-2, SHA-3) drastically reduces the practical risk.

Bit length is critical: the longer the hash, the more secure and collision-resistant it is.

Message Integrity Using Hash Functions
Scenario

Alice wants to send a message to Bob.

Alice is concerned only with integrity, not confidentiality (no need to hide the message from eavesdroppers).

Process

Hashing by Alice

Alice inputs the message into a hashing algorithm.

The algorithm produces a hash value (fingerprint) of the message.

Alice sends both the original message and the hash to Bob.

Alice also informs Bob of which hash algorithm was used.

Verification by Bob

Bob receives the message and the hash.

Bob runs the same hashing algorithm on the received message.

Bob compares the calculated hash with the received hash:

If they match: The message is unaltered.

If they differ: The message was tampered with during transit.

Important Notes

Alice and Bob do not manually handle the hashing or algorithm exchange; software automatically manages this.

Hashing ensures that any modification to the message (even a single bit) produces a completely different hash, allowing tampering detection.

This approach verifies integrity, but does not provide confidentiality—the message is readable by anyone who intercepts it.

Digital Signatures
Concept

Digital signatures are the electronic equivalent of handwritten signatures on physical documents.

Purpose:

Identify the signer uniquely

Confirm agreement or attestation

Handwritten signatures can be forged easily; digital signatures aim to be unique and hard to forge.

How It Works

Signing the Data

The signer uses their private key to “encrypt” a hash of the message (in RSA terminology, this generates the signature).

Since the private key is unique to the signer, the signature unmistakably links the signer to the data.

Verifying the Signature

The recipient uses the signer’s public key to decrypt and verify the signature.

Successful verification confirms:

Authenticity: The message is truly from the signer.

Integrity: The message has not been altered.

Non-repudiation: The signer cannot deny having signed the message.

Digital Signature Example
Scenario

Alice wants to send a message to Bob.

Goals:

Ensure message integrity (message hasn’t been altered).

Ensure authentication and non-repudiation (Bob can verify it came from Alice).

Step-by-Step Process

Hashing the Message

Alice creates a hash of the message using a hashing algorithm (e.g., SHA-2).

This hash is a unique fingerprint of the message.

Signing the Hash

Alice uses her private key to encrypt the hash, creating a digital signature.

The signature binds Alice's identity to the message because only she possesses her private key.

She then sends:

The original message (plaintext)

The digital signature (encrypted hash)

Verifying the Signature

Bob receives the message and the signature.

Bob uses Alice's public key to decrypt the signature, retrieving Alice's hash.

Bob hashes the message he received (using the same algorithm, SHA-2) to generate his own hash.

Bob compares his hash with the decrypted hash:

If they match:

The message was not altered.

The message was authentically sent by Alice.

If they do not match: The message may have been tampered with or the signature is invalid.

Steganography (Stego)
Definition

Steganography literally means “covered writing.”

It is the practice of hiding data within another medium so that the existence of the hidden data is concealed.

Different from encryption, which hides the content of a message, steganography hides the fact that a message exists.

History

Dates back to Ancient Greece.

Modern electronic steganography is relatively new, using digital media as carriers.

Carrier Media

Hidden data can be embedded in:

Images: bmp, png, gif, jpg

Documents: Word files, text files

Other digital media: machine-generated images (e.g., fractals), audio files

Applications

Conceal sensitive information during transmission or storage.

Disguise encrypted data to prevent detection or legal restrictions on encryption.

Steganography complements encryption when secrecy of communication is paramount.

Crypto versus Steganography
Cryptography (Crypto)

Purpose: Provides confidentiality, integrity, and non-repudiation.

Visibility: Encrypted data is easily detectable, even if unreadable.

Limitation: Does not hide the fact that communication or data exists.

Example: Ciphertext in emails, secure file storage.

Risk: Encrypted files can mark targets for attackers (like valuables in a vault).

Inference Attacks: Observers may infer sensitive activity even without seeing content.

Steganography (Stego)

Purpose: Provides secrecy by concealing the existence of data.

Visibility: The hidden message is not obvious; the carrier looks normal.

Examples:

Hiding messages in images, audio, or documents.

Concealing encrypted data inside a normal-looking file.

Limitation: Stego alone does not guarantee data safety.

The hidden data still depends on encryption strength.

Weak or absent encryption makes stego vulnerable if discovered.

Detecting Cryptography
Both humans and computers can easily detect encryption because encryption increases file size and

mathematically normalizes the occurrence of data. When viewing a document or email comprised of garbled
characters, one might infer that it is actually encrypted

Histograms

Histograms are graphical representations of the number of occurrences of data in a given distribution of such
data. For example, a histogram of a text document would show the number of occurrences of each character
that appears in the document. A normal text document would generate a histogram that shows that the
frequency of characters varies greatly. In a histogram for an encrypted document, the frequency of characters
is normalized because good crypto always produces as random of ciphertext as possible. The same factor that
helps prevent encryption from being interpreted makes it easier to detect.

How Steganography Works (Super Simple)

What is Steganography?

Steganography = hiding data inside other data

The goal is to hide the message so no one knows it exists

Main Parts of Steganography

1 Host (Carrier)

This is the file that hides the message

Usually looks normal and harmless

Examples:

Images (JPG, PNG, BMP)

Audio files

Videos

Word or text files

HTML files

People see the file, but don’t know it contains hidden data

2Hidden Message

This is the secret data

Can be:

Text

Another file

Encrypted data

How the Message Is Hidden

The message is placed in unused or less noticeable parts of the file

Examples:

Tiny color changes in an image

Extra bits in audio or video

Spaces or formatting in text files

The file still works normally and looks unchanged

General Types of Steganography (Easy Version)

There are 3 main ways to hide a message using steganography:

1 Injection

Add the secret data into the file without changing the original content

The hidden data is inserted into unused or ignored parts of the file

The original file looks and works the same

Simple example:

Add secret data at the end of an image file

Many programs ignore extra data, so the image still opens normally

File looks normal, extra data is just “injected”

2Substitution

Replace small, unnoticeable parts of the file with secret data

Tiny pieces of the file are changed

Changes are so small humans cannot notice

Simple examples:

Change the least significant bit (LSB) of image pixels

Slightly change sound frequencies in audio files

Most common stego method

3 File Generation

Create a brand-new file that contains the hidden message

The file is generated from scratch

The hidden message is built into the file’s structure

Simple examples:

A generated image or fractal that secretly contains data

A text file where spacing or formatting encodes the message

Nothing to compare against—file looks completely normal

Injection (Steganography) – Simple Explanation

Injection stego hides secret data by putting it into parts of a file that are ignored when the file is opened or used.

The file still works normally, but extra hidden data is inside it.

How Injection Works (Plain Language)

Many file types have unused areas (“holes”)

Programs ignore these areas

We insert secret data into those ignored areas

Simple Examples
HTML Files

Browsers ignore:

Comments

Hidden form fields

Secret data can be placed there

Website looks normal, but data exists in the file

Images / Audio (GIF, MP3, etc.)

These files allow comments

Comments are not shown when viewing or playing the file

Secret data can be stored inside comments

Word Documents

Create a big document → save it

Delete a lot of text

File size stays large

The unused space can hide data

Key Characteristics of Injection Stego

✔ File looks normal
✔ Original content is unchanged
✔ Secret data is stored in unused areas
❌ File size increases

Main Weakness

File size becomes larger than expected

Easy to detect if:

Original file size is known

File size looks unusual for its type

Example:

A small Word document that is hundreds of MB → suspicious

How Substitution Works (Plain Language)

The file already has data (image pixels, sound samples, etc.)

Some of this data is not noticeable to humans

We replace that unimportant data with secret bits (0s and 1s)

Example:

Original bits: 1010
Hidden bits: 0110
→ Replace small parts

Common Example: Images (LSB Method)

Images store colors using bits

The least significant bit (LSB) barely affects the color

Changing it:

Human eye doesn't notice

Data can be hidden safely

✔ Image looks the same
✔ Secret data is inside

Key Characteristics

✔ File size stays the same
✔ Very hard to detect visually
✔ Most commonly used stego method
✕ Limited space for hidden data
✕ Too much data = file quality loss

Why It Can Cause Damage

If too much data is hidden:

Image becomes blurry

Audio sounds distorted

That's why only insignificant data is replaced

Limits of Substitution

You can only replace existing bits

Once you overwrite too much:

File quality degrades

Attackers may notice

Generate a New File (Steganography)

Generate-a-New-File stego hides data by creating a completely new file from the secret message itself.

Unlike other stego methods:

No existing host file is required

The stego tool creates the carrier automatically

How It Works (Simple)

You start with secret data (text, message, file)

A stego program uses that data as input

The program generates a new file that looks innocent

The hidden data is embedded in how the file is generated

Detecting Steganography (Simple Explanation)

Steganography detection is hard.
There is no single or universal way to detect stego in files.

Why Detection Is Difficult

Stego can be done in many different ways

Each tool hides data differently

What looks “normal” for one file may look suspicious for another

Unlike encryption, stego tries to look normal

Detection usually works tool-by-tool, not generically.

Encryption vs Steganography Detection
Type Detection Method

Encryption High entropy / flat histogram
Steganography File abnormalities (harder to prove)

✔ Encrypted files are easy to notice
✕ Stego files try to blend in

Common Stego Detection Method (Images)

Most steganography hides data in images, especially:

PNG

BMP

Least Significant Bit (LSB) Analysis

Images store color using bits

LSB = smallest color impact

Stego tools overwrite LSBs to hide data

Image looks the same to humans

Detection tools analyze LSB patterns

Look for abnormal or non-random changes

Steganography Detection Tools

Common tools include:

StegExpose

StegSecret

These tools:

Analyze images

Look for LSB irregularities

Flag suspicious files

Important:

Abnormalities ≠ proof of steganography

Key Limitations

No universal detection method

Abnormal files may be false positives

Stego detection requires:

Knowledge of file formats

Knowledge of hiding techniques

Specialized tools

One-Way Function (Easy Explanation)

Easy to compute in one direction

Extremely hard to reverse

Impossible to solve quickly (not polynomial time)

Example idea:

Easy: Lock a box

Hard: Break the lock

Easy again: Open it with the key (trapdoor)

Trapdoor

The trapdoor is secret information

It allows the hard problem to be solved easily

In cryptography:

Public key → used to encrypt

Private key → trapdoor used to decrypt

Why This Is Important

If attackers:

Do not have the trapdoor

Must solve a very hard math problem

They cannot realistically break the encryption

Cryptography Goals (Why We Do This)

Using these math ideas, we can build systems that provide:

Confidentiality – only authorized users can read data

Integrity – data cannot be changed without detection

Authentication – prove who sent the data

Non-repudiation – sender cannot deny sending it

Role of Mathematics

Cryptography uses many math fields, such as:

Probability

Complexity theory

Finite fields

Abstract algebra

These help create strong and secure algorithms

Concepts in Cryptography (2) — Super Easy Version
The main idea (1 sentence)

Cryptography works because some math problems are too hard for computers to solve.

Easy vs Hard Problems (Real Life Example)
Easy problem

Counting coins in your pocket

A computer can do this very fast.

Hard problem

Guessing a password by trying every possible combination
This can take years or forever.

Cryptography uses hard problems, not easy ones.

What Does “Big-O” Mean? (Very Simple)

Big-O just answers this question:

If the input gets bigger, how much slower does the computer become?

We don’t care about exact time — only how fast the time grows.

Easy Time (Good for Attackers ✕)

O(1) → Always fast

O(n) → Bigger input → a bit slower

$O(n^2)$ → Bigger input → much slower

Computers can still solve these.

Used in:

Symmetric encryption (like AES)

Hard Time (Good for Security ✓)

$O(2^n)$ → Explodes very fast

Takes longer than human lifetime

Sometimes longer than the age of the universe

Computers cannot realistically solve this.

Used in:

Asymmetric encryption (public-key crypto)

Why Cryptography Loves Hard Problems

Imagine this:

Locking a door is easy

Breaking the lock without a key is very hard

That’s exactly how cryptography works.

Encrypting = easy

Decrypting without key = almost impossible

“Intractable” (Easy Meaning)

Intractable = Too hard to solve in a reasonable time

Even with powerful computers.

Important Warning

We believe these problems are hard

But math evolves

A hard problem today might become easy tomorrow

That’s why:

Crypto algorithms must be updated over time

Famous Hard Problems (No Math Needed)

1 Factoring big numbers

Easy to multiply

Extremely hard to break apart

Used in RSA

2 Discrete logarithms

Used in ElGamal

3 Elliptic curves

Smaller keys, same security

Used in ECC

Concepts in Cryptography (3) — Very Easy Explanation
The main idea

Cryptography uses math problems that are easy to do one way, but extremely hard to reverse.

What Is the Problem Here?
The problem:

Factoring a very large number into prime numbers

Example:

Small number:
15 = 3 × 5 → easy ✅

Very large number (hundreds or thousands of digits):
Extremely hard ❌

Why Is This Problem "Hard"?

Because:

There is no fast method to break big numbers into primes

The only known way is basically:

Try again and again until something works

For small numbers → fast
For huge numbers → can take years or longer

Why Cryptography Uses This

This problem has special properties:

1Easy one way

Multiply two large primes → easy

2Hard the other way

Given the result, find the two primes → very hard

Perfect for security

Is This Proven to Be Hard?

No mathematical proof yet

But:

Experts have studied it for decades

No efficient solution found

Widely accepted as intractable

So we trust it.

Where Is This Used?
RSA encryption

One of the most common public-key systems

Used in:

HTTPS

Secure email

Digital certificates

RSA security = factoring is hard

Why Does the Slide Mention DES?

RSA is about 1000 times slower than DES

DES (symmetric encryption) is fast

RSA (asymmetric encryption) is slow but secure for key exchange

That’s why we:

Use RSA to exchange keys

Use AES/DES to encrypt actual data

Why Is RSA Still Considered Secure?

Because:

Factoring large numbers is still impractical

Proper key sizes make attacks unrealistic

Used worldwide (de-facto standard)

What About Patents?

RSA was patented in the past

Patent expired in 2000

Now anyone can use it freely

Here's Concepts in Cryptography (4) explained in a much simpler way:

Main Idea

Just like factoring large numbers is hard (RSA), another hard math problem is called the Discrete Logarithm Problem (DLP).
This problem is used to make cryptography secure.

What is the Discrete Logarithm Problem?

We want to solve equations like:

m
o
d

=

a
x
modn=b

,

,

a,n,b → known numbers

x → unknown number we want to find

✅ Easy: Calculate

b if you know

x
❌ Hard: Find

x if you only know

b

Small Example

Equation:

8

m
o
d

7
=
1
8
x
mod7=1

Easy to calculate:
8
3
=
512
8
3
=512,

512
m
o
d

7
=
1
512mod7=1 ✅

Hard to find

x just by looking at 1 without trying numbers repeatedly

Why Is It Hard?

The difficulty increases drastically with hundreds- or thousands-bit numbers

No fast formula to find
x

Only method is trial-and-error, which is impossible for very large numbers

Why Do Cryptographers Like It?

Because it's intractable (very hard to solve), it can be used to build secure systems, just like RSA:

ElGamal encryption → encrypts messages

Diffie-Hellman → securely exchange keys

Schnorr signature → digital signatures

DSA (Digital Signature Algorithm) → official standard for signing

Is It Proven to Be Hard?

No formal proof exists

But decades of research suggest it cannot be solved quickly (polynomial time)

Trusted like factoring large numbers

Here's Concepts in Cryptography (5) explained more simply:

Main Idea

Another hard math problem is like the discrete logarithm problem, but now applied to elliptic curves. This is the basis of Elliptic Curve Cryptography (ECC).

ECC is very popular because it is secure, fast, and efficient.

What Makes It Hard?

Similar to the regular discrete logarithm problem (DLP)

But instead of working with numbers in a finite field, we work with points on an elliptic curve

Finding the “private key” from a point on the curve is extremely difficult, even with huge computing power

Why Use ECC?

Higher security with shorter keys → smaller keys are harder to guess

Faster computation → good for mobile devices and smart cards

Lower power and storage needs → ideal for IoT devices

Real-World Examples

Elliptic Curve ElGamal → encrypting messages

Elliptic Curve Diffie-Hellman (ECDH) → exchanging keys securely

Elliptic Curve Schnorr → digital signatures

Elliptic Curve DSA (ECDSA) → standard for signing documents

Analogy

Think of ECC like a supercharged version of the discrete logarithm problem:

Hard to solve (private key)

Easy to do in one direction (generate public key)

Works well on devices that need speed and low powe

DES (Symmetric)
Data Encryption Standard
Released March 17, 1975
Rather fast encryption algorithm
Widely used; a de facto standard
Symmetric key, 64-bit block
cipher
56-bit key size: Small 256 keyspace
Today, DES is not considered

DES (Symmetric)
DES used to be one of the more commonly used encryption algorithms in the world, which is based on IBM's
Lucifer cipher.
Because of the internal bit-oriented operations in the design of DES, software implementations were slow,
whereas hardware implementations were faster. There are four different DES operation modes for use:
Electronic codebook (ECB) mode, cipher block chaining (CBC) mode, output feedback (OFB) mode, and
cipher feedback (CFB) mode

Why is DES Weak?
1Small Key Size

DES uses a 56-bit key.

That means there are 2^{56} possible keys.

Today's computers can try all keys very fast.

This is called a brute-force attack.

2 DES Can Be Cracked Quickly

In 1997, DES was publicly cracked in the RSA Challenge.

It took only 5 months back then.

Today, it can be cracked in hours or minutes with modern hardware.

So DES is no longer secure.

3 The Algorithm Isn't Broken — The Key Is Too Short

Important point

DES math itself is not flawed.

The problem is that the key size is too small.

Short keys = easy to guess.

4Using DES Multiple Times?

One idea was to encrypt the data more than once (example: encrypt → encrypt again).

This can increase security only if the cipher is not a "group".

Even then, it raises questions:

How many times should we encrypt?

Is it still efficient?

This led to Triple DES (3DES) as a temporary fix.

First: What does "group" mean here?

In simple terms:

If an encryption algorithm is a group, then:

Encrypting twice with two keys
gives the same result as encrypting once with one key.

Formula (don't panic):
E(K2, E(K1, M)) = E(K3, M)

That means:

Two encryptions = one encryption

So encrypting multiple times is useless

Why is this bad?

If an algorithm is a group:

Encrypting data 2 times

or 3 times

or 100 times

does NOT increase security

Attackers only need to break one encryption.

Good News : DES Is NOT a Group

In 1992, researchers proved:

DES is NOT a group encryption

This means:

Encrypting with DES multiple times actually helps

Each encryption adds more complexity

Multiple encryptions are not equivalent to one encryption

Why This Matters

Because DES keys are small (56-bit), DES alone is weak.

But since DES is not a group:

We can encrypt more than once

This increases security

That's why Triple DES (3DES) exists:

Encrypt → Decrypt → Encrypt

Uses multiple keys

Much harder to brute-force than DES

What is Triple DES (3DES)?

Triple DES = DES used 3 times

Instead of encrypting data once with DES, we do it three times using different keys.

This makes DES much harder to break.

How 3DES Works (Very Simple)

Usually done like this:

Encrypt with Key 1
Decrypt with Key 2
Encrypt with Key 3

This is called EDE mode (Encrypt–Decrypt–Encrypt).

Why decrypt in the middle?

For compatibility with old DES systems

Still gives strong security

Key Length (Easy Explanation)

DES → 56-bit key ❌ (too weak)

3DES (2 keys) → 112-bit security

3DES (3 keys) → 168-bit security

Bigger key = harder to crack

Why Use 3DES Instead of DES?

DES is broken by brute force

3DES is much stronger

Uses the same DES algorithm

Only needs a software upgrade, not new hardware

✅ That made it very popular in banks and legacy systems

Is 3DES Secure?
Good news

No public reports of fully cracking 3DES

Heavily analyzed by cryptography experts

Still used in some systems today

Bad news

Slow (3× the work)

Considered legacy

New systems prefer AES

Why 3DES Resists Attacks
Meet-in-the-Middle Attack?

Works well on Double DES

Fails on Triple DES

Third encryption breaks the attack logic

That's why 3DES exists.

What is AES?

AES (Advanced Encryption Standard) is:

A symmetric encryption algorithm

The modern replacement for DES and 3DES

Fast, secure, and widely used

Used in:

HTTPS

Wi-Fi (WPA2 / WPA3)

Disk encryption

VPNs

Big Picture (Simple)

AES works on data in blocks (128 bits).

You can imagine the data as a 4×4 table of bytes, called the State.

AES secures the data by repeating 4 simple steps many times.

The 4 AES Steps (Very Easy)
1AddRoundKey

Mix data with the secret key

Uses XOR

Combines the data with the round key

Without the key → nothing works

Think: “Lock the data using the key”

2 SubBytes

Change each byte

Every byte is replaced using a lookup table (S-box)

Makes patterns disappear

Think: “Replace each letter with a secret symbol”

3 ShiftRows

Shuffle rows

Row 1 → shift left by 1

Row 2 → shift left by 2

Row 3 → shift left by 3

Think: “Move characters around so they don’t stay in place”

4 MixColumns

Mix data vertically

Each column is mathematically mixed

Spreads changes across the whole column

Think: “Shake the columns so everything is mixed together”

AES (Advanced Encryption Standard) — Easy Explanation

What is AES?

AES is a modern symmetric encryption algorithm used to keep data private and secure.

✅ Where AES Is Used (USAGE)

AES is everywhere today:

Wireless security (WPA2, WPA3 Wi-Fi)

Full disk encryption (BitLocker, FileVault)

VPNs

HTTPS / TLS

Mobile phones and apps

If data is important, AES is usually protecting it.

Security of AES (VULNERABILITIES)

❌ No real-world cryptographic weaknesses found

The only realistic attack is:

Brute-force guessing the key

Why brute force doesn’t work in practice:

AES keys are 128, 192, or 256 bits

That’s too many possibilities

Even supercomputers would need billions of years

So in real life, AES is considered secure.

Why AES Is Trusted

Designed through an open competition

Reviewed by cryptographers worldwide

Studied for years before adoption

Any weakness would likely already be known

This public review is why AES is strong.

Important Note (Very Simple)

AES itself is secure, but problems happen when:

Weak passwords are used

Bad modes are chosen (like ECB)

Keys are stolen

These are implementation mistakes, not AES problems.

RSA (Asymmetric Encryption) — Easy Explanation

What is RSA?

RSA is an asymmetric (public-key) encryption algorithm.
It uses two keys:

Public key → shared with everyone

Private key → kept secret

✅ Where RSA Is Used (USAGE)

RSA is widely used on the Internet, especially for:

SSL / TLS (HTTPS websites)

Secure key exchange

Digital signatures

Web browsers and servers

RSA became very popular after its patent expired in 2000, allowing free use.

Security of RSA (VULNERABILITIES)

Important point:

RSA itself has NOT been broken

However, problems happen when:

❌ Small key sizes are used (e.g., old 512-bit keys)

❌ Bad implementations

❌ Poor random number generation

❌ Private key leakage

In almost all real attacks, the implementation was weak, not RSA.

Why RSA Is Secure

Based on the difficulty of factoring very large numbers

Easy to encrypt with the public key

Extremely hard to derive the private key

Security depends on:

Private key secrecy

Large key sizes (2048 bits or more)

Elliptic Curve Cryptosystem (ECC) — Easy Explanation

What is ECC?

ECC is an asymmetric (public-key) cryptography system, like RSA, but more efficient.

It uses:

Public key → shared

Private key → secret

ECC is based on hard math problems using elliptic curves, which are extremely difficult to solve.

✅ Where ECC Is Used (USAGE)

ECC is used when systems need:

⚡ High speed

Low power consumption

Low storage usage

Strong security with small keys

Common places:

Mobile phones

Wireless networks

Smart cards

IoT devices

Modern SSL/TLS connections

ECC is growing in popularity and replacing RSA in many cases.

Why ECC Is Strong

Small keys, same security

ECC 256-bit ≈ RSA 3072-bit

Faster computations

Less battery usage

Less memory needed

This is why ECC is ideal for mobile and embedded systems.

ECC Vulnerabilities

Important to know:

❌ ECC itself has NOT been cracked

Real problems come from:

Bad implementations

Weak or poorly chosen curves

Side-channel attacks

Poor random number generation

Attacks target how ECC is used, not the ECC math itself.

Comparing Key Lengths — Easy Explanation

1 Same Algorithm → Bigger Key = Stronger Security

Inside one algorithm (like AES):

128-bit < 192-bit < 256-bit

Bigger key = harder to crack

✅ This rule is simple and true.

2 Different Algorithms → Key Sizes Are NOT Equal

When comparing different types of cryptography, key length numbers do NOT mean the same thing.

A 256-bit key in one algorithm can be much stronger than a 256-bit key in another.

Symmetric vs Asymmetric (Very Important)
Symmetric Cryptography (AES, DES)

Same key for encryption and decryption

Fast

Very strong per bit

Shorter keys are enough

Typical key sizes:

40–256 bits

Asymmetric Cryptography (RSA, ECC)

Public key + Private key

Slower

Weaker per bit

Needs much larger keys for same security

Typical key sizes:

1000 bits or more

Why Asymmetric Keys Are Larger

Asymmetric algorithms are:

More complex

Easier to attack per bit

To stay secure, they increase key size

Larger keys = more work for attackers

Simple Strength Comparisons (from NIST)

Symmetric / ECC	Equivalent RSA Key Size
3DES (112-bit)	RSA 2048-bit
AES 192-bit	RSA ~7680-bit
ECC 256-bit	RSA 3072-bit

Same security, very different key lengths

☆ Why ECC Is Special

ECC is asymmetric, but:

Needs only about double the symmetric key size

Much smaller than RSA

That’s why ECC is popular in:

Mobile devices

Wireless systems

IoT

What Is Hashing? (First, very simple)

Hashing takes any message (short or long)

Produces a fixed-length output called a hash

The hash is like a digital fingerprint

Same message → same hash
Change 1 letter → totally different hash

What Hashing Is Used For

Integrity → check if data was changed

Password storage

Digital signatures (hash first, then sign)

Hashing is NOT encryption

You cannot reverse a hash to get the message back

Example (Simple)

Message:

"Message"

After SHA-256:

c7cfaff189585980c90c54be0b56980e...

Input: any length

Output: fixed length

SHA Family Overview

SHA = Secure Hash Algorithm (by NIST)

SHA-1 (Deprecated ❌)

Output: 160 bits

Old and no longer secure

Vulnerable to collisions

❌ Do NOT use

✅ SHA-2 (Secure & Widely Used)

Common versions:

Algorithm	Output Size
SHA-256	256 bits
SHA-512	512 bits

✔ Very secure
✔ Used in:

TLS

Password hashing (with salt)

Digital signatures

SHA-3 (Newest Generation)

Algorithm	Output Size
SHA3-256	256 bits
SHA3-512	512 bits

✔ Different internal design from SHA-2

✔ Protection against future weaknesses
✔ Not broken

Why New SHA Versions Exist

Attackers try to find collisions

Two different messages → same hash

When weaknesses appear:

New SHA versions are created

SHA-2 and SHA-3 are responses to SHA-1 weaknesses

What Is Cryptanalysis? (Very Simple)

Cryptanalysis = trying to break or weaken encryption

Before we trust any crypto system, experts must:

Attack it

Test it

Try to break it

If it survives many attacks → we trust it

What Do Attackers Target?

Cryptanalysts don't only attack the algorithm, they also attack:

Math behind the algorithm

Key size and key generation

Key storage and protection

Implementation bugs

Hash collisions

System configuration

Weak implementation can break strong crypto

Different Crypto = Different Attacks

Crypto Type	Attacks Focus On
Symmetric	Keys, math, patterns
Asymmetric	Math problems, key size
Hashing	Collisions, weaknesses

Main Categories of Cryptanalytic Attacks (Easy)

1 Analytic Attacks

Use math and algorithms

Try to:

Find the key

Reduce how many keys need testing

Example: solving equations instead of brute force

2Statistical Attacks

Look for patterns

Use frequency and statistics

Works well when data is predictable

Example: letters in English appear at different rates

3 Differential Attacks

Compare small changes in input

Watch how output changes

Mostly used on symmetric encryption and hashes

Example: change 1 bit → see how ciphertext changes

4 Linear Attacks

Look for linear relationships

Compare plaintext + ciphertext pairs

Also mainly for symmetric encryption

5 Differential-Linear Attacks

Combine:

Differential attacks

Linear attacks

Stronger than using one alone

Key Point to Remember

Crypto is trusted only after it survives years of cryptanalysis

No crypto is trusted just because:

It is new

It is secret

It “looks complex”

The Birthday Paradox (Simple Example)

You might think:

“To match my birthday, someone must be born on the same day as me → 1 out of 365.”

That’s true for one specific person.

BUT…

If you put 23 people in a room, there is more than a 50% chance that any two people share a birthday.

Why?

Everyone is compared with everyone else

Many comparisons happen at the same time

Key idea:
We are not looking for a specific match — just any match.

How This Applies to Hashing

Hash functions turn messages into fixed-size values

Sometimes, two different messages produce the same hash

This is called a hash collision

Just like two people sharing a birthday.

The Birthday Attack in Cryptography

An attacker:

Generates many different messages

Hashes all of them

Looks for any two messages with the same hash

Once a collision is found:

One message can be replaced with the other

Integrity is broken

This is dangerous for:

Digital signatures

Password hashes

Integrity checks

1Protecting Data in Transit

What it is: Data traveling across a network (like the internet or internal company network).

Risk: Hackers can intercept and read or modify your messages (man-in-the-middle attacks).

Solution: Use VPNs (Virtual Private Networks) or encrypted communication protocols (SSL/TLS).

Example: When you send an email through Gmail, SSL/TLS encrypts it so attackers cannot read it.

2 Protecting Data at Rest

What it is: Data stored on disks, drives, or cloud storage.

Goal: Keep sensitive information safe even if someone steals the storage device.

Methods:

Data encryption: Encrypt files individually.

Full disk encryption: Encrypts the whole hard drive (everything is protected).

Tools: GNU Privacy Guard (GPG) for encrypting files and emails.

Example: Encrypting your laptop with BitLocker or VeraCrypt.

3 Key Management

Why it matters: Cryptography is only as strong as the way you handle keys.

How it works:

Public Key Infrastructure (PKI): Framework for issuing, storing, and managing cryptographic keys.

Digital Certificates: Prove identity online (used in websites, emails).

Certificate Authorities (CA): Trusted entities that issue certificates.

Example: When you visit a website starting with https://, your browser checks a certificate issued by a CA to ensure the site is legitimate.

1 Local vs. Wide Networks

LAN (Local Area Network): All devices are close together (same building or floor).

Easier to control and secure because you own everything.

PAN (Personal Area Network): Very small network, like your phone, laptop, and smartwatch.

When networks start spreading across different locations, you may need third-party infrastructure (like a telecom company) to connect them.

2 Third-Party Infrastructure

If you rely on a telecom provider:

They can technically intercept your data, because it travels over their equipment.

In the past, companies didn't worry about this — they assumed the connection was private.

3 Private Lines and Leased Lines

Leased line: A dedicated line provided by a telecom for your exclusive use.

Pros: Connects multiple offices securely without installing your own cables.

Cons: Expensive, billed monthly, costs increase with distance.

These lines were considered private networks, so people often skipped extra encryption.

4 Modern View

Today, even if using leased lines or MPLS (modern equivalent), companies consider the telecom a potential threat.

Encryption is necessary to protect data in transit, even over private networks.

Same methods used for internet encryption (VPNs, TLS, etc.) can be applied to private networks.

✅ Key Takeaways

Ownership matters: The more you own/control the network, the easier it is to secure.

Private lines = dedicated, but not automatically secure.

Modern security assumes any third party could be a threat, so always encrypt sensitive data in transit.

Methods for protecting data in transit are consistent across the internet and private networks.

What a VPN Does

VPN (Virtual Private Network) creates a secure “tunnel” for your data over an untrusted network (like the internet).

How it works:

Data starts as cleartext at your device.

VPN encrypts it into ciphertext.

Ciphertext travels over the network (anyone sniffing it sees only gibberish).

VPN decrypts it back into cleartext at the receiving end.

Why It’s Called “Virtual”

VPN isn’t a separate physical network — it’s a “virtual” network layered on top of any existing network (internet, leased lines, etc.).

It provides end-to-end protection, even if the underlying network is untrusted.

VPN vs. Private Network

Expensive private lines (leased lines) can connect offices securely, but cost a lot.

VPNs can achieve similar confidentiality without requiring expensive physical circuits.

Example uses:

Connecting remote offices securely over the internet.

Allowing remote employees to access internal resources safely.

Key Takeaways

VPN = encrypted tunnel for data in transit.

Works over any network, not just the internet.

Encrypts sensitive data to prevent interception.

Cost-effective alternative to leased/private lines.

VPNs are flexible
A VPN "tunnel" over the Internet can be set up rapidly; a private network can take weeks (or months)
SEC401 | Security Essentials: Network, Endpoint, and Cloud
VPN Advantage: Flexibility
One of the biggest benefits of VPN technology is its flexibility. If you need a secure channel between two hosts only for a day, or even an hour, then a VPN might fit the bill. After you have all the components to establish a VPN, setting one up requires only configuration. This makes the technology far more flexible than private circuits, which must be ordered far in advance of their use and might also require additional hardware.
This flexibility lends itself to creating new business solutions.

What is a VPN (in simple words)?

A VPN (Virtual Private Network) lets you use the normal internet, but in a secure and private way.

It creates a secure tunnel between you and another network, and all data inside this tunnel is encrypted.

Why do people use VPNs?
Advantages

Cheap and easy

You don’t need expensive private lines.

You just use normal internet.

Secure (Encrypted)

Your data is protected from hackers and spying.

Good bandwidth for low cost

Usually faster and cheaper than private networks.

Good for:

File transfers

Email

Remote work

Disadvantages

No guaranteed speed

Internet speed depends on traffic and congestion.

Delay (Latency)

Usually 1–2 seconds delay.

This can be a problem for:

Financial systems

Medical systems

Real-time systems

Not good for time-critical systems

If timing is very important, VPN is risky.

Types of VPN
1. Client-to-Site VPN

Example:
An employee connects from home to the company network.

Laptop → VPN Server → Company network

Data is encrypted until it reaches the company.

2. Site-to-Site VPN

Example:
A branch office connects to the main office.

Network → VPN Gateway → Internet → VPN Gateway → Network

Used between two locations.

3. Client-to-Client (End-to-End VPN)

Example:
Two computers connect directly with encryption.

Most secure type.

But hard to manage.

Needs encryption keys for every device.

Why is Client-to-Client rare?

Because:

Every device needs its own keys.

Too many devices = too much management.

Requires PKI (certificate systems).

Easy analogy

Internet = public road

VPN = armored car

Dedicated line = private tunnel

IPsec Overview (Simple)
What is IPsec?

IPsec (Internet Protocol Security) is a standard used to make network communication secure.

It was created by IETF and is an open standard, so it works with devices from different companies (Cisco, Fortinet, Windows, Linux, etc).

Why do we need IPsec?

Because:

Normal internet protocols are not secure by default.

They don’t protect:

Data privacy

Data integrity

Authentication

So IPsec was created to add security to IP traffic.

What can IPsec do?

IPsec provides:

Confidentiality → Encrypts data

Integrity → Makes sure data is not changed

Authentication → Verifies who sent the data

Anti-replay → Prevents re-sending old packets

Why is IPsec popular?

Works with most VPN devices

Fits inside network infrastructure

Works for small and large networks

Very common in site-to-site VPNs and remote access VPNs

Types of IPsec Headers

IPsec has two main protocols:

1. Authentication Header (AH)

AH provides:

Integrity → Data not modified

Authentication → Confirms sender

No encryption → Data is still visible

What AH does simply:

It adds a digital fingerprint (hash) to each packet.

If someone changes anything:

Source IP

Destination IP

Data

→ The fingerprint changes → packet is rejected.

So AH protects against:

Packet modification

IP spoofing

Replay attacks

But anyone can still read the data.

2. Encapsulating Security Payload (ESP)

ESP provides:

Integrity

Authentication

Confidentiality (encryption)

This is the most used IPsec protocol.

What ESP does simply:

It:

Encrypts the data

Adds integrity check

Confirms sender

So attackers:

Can’t read the data

Can’t modify it

Can’t fake sender

Very simple analogy

Imagine sending a letter:

AH:

Letter is open

But has a tamper-proof seal

ESP:

Letter is sealed and locked

Only the receiver has the key

Why ESP is preferred over AH in real networks
Important point first:

When ESP calculates integrity and authentication:

It does NOT use the source or destination IP addresses.

It doesn't matter if the data is encrypted or not.

The security check is always the same.

But AH DOES use the IP addresses.

This is the main difference.

Why do both AH and ESP support authentication?

People often ask:

Why do we need authentication in both AH and ESP?

The simple answer:
Because in some real situations, AH does not work properly, but ESP does.

How AH works (the problem)

AH creates something called an ICV (Integrity Check Value) using:

Source IP

Destination IP

Packet data

Other fields

If any of these change, the packet is rejected.

This is fine only if IP addresses never change.

But in real life, they do change.

Types of IPsec Modes

IPsec can work in two modes:

Transport Mode

Tunnel Mode

Both AH and ESP can use these modes.

1. Transport Mode (Host ↔ Host)
What it does:

Protects communication between two specific computers.

The original IP header stays.

IPsec header is added after it.

Used for:

Client-to-client

Client-to-site

What is protected:

Only the data (payload).

The IP addresses are still visible.

2. Tunnel Mode (Gateway ↔ Gateway)
What it does:

Protects communication between two networks.

The entire original packet is put inside a new one.

A new outer IP header is added.

Used for:

Site-to-site VPN

What is protected:

Original IP header

Original data

Everything inside the tunnel

To the original packet:

The whole internet looks like one single hop.

AH in Tunnel Mode (simple)

Gateway takes original packet.

Wraps it inside a new packet.

AH adds integrity check.

No encryption.

So:

Data cannot be modified.

But still readable.

ESP in Tunnel Mode (most common)

Gateway wraps original packet.

ESP encrypts the inside.

Adds authentication.

So:

Data is hidden.

Data is protected.

Data is verified.

This is how real VPNs work today.

Security Associations (SA)

Before IPsec works, both sides must agree on:

Which protocol (AH or ESP)

Which encryption

Which keys

Which algorithms

This agreement is called a Security Association (SA).

Important facts:

SA is one direction only

For two-way traffic → need two SAs

IKE (Internet Key Exchange)

IKE is the system that:

Creates the SAs

Exchanges keys

Sets security rules

Without IKE:

IPsec cannot start.

IKE Phases
Phase 1

Creates a secure channel for negotiation.

Result:

ISAKMP-SA (protects future talks)

Modes:

Main Mode → hides identities

Aggressive Mode → faster, less secure

Phase 2 (Quick Mode)

Negotiates:

Actual IPsec SAs

Keys for ESP/AH

Rekeying when keys expire

This is where:

Real VPN security is built.

What is an SSL VPN?

An SSL VPN is a type of VPN that uses web technologies (like HTTPS) instead of IPsec.

In most cases:

All you need is a web browser to use it.

No special VPN software is required.

Why do companies use SSL VPNs?

Because IPsec is complex:

Hard to configure

Hard to manage on user devices (especially BYOD)

SSL VPNs are:

Easier to deploy

Cheaper to operate

Fewer configuration problems

Is SSL VPN secure?

Yes, cryptographically it is as strong as IPsec.

Important note:
Even though it's called SSL VPN, it actually uses TLS, not the old insecure SSL.

So:

The name is old, the security is modern.

Main risk of SSL VPNs

Because they use web browsers:

If the browser has vulnerabilities → VPN is affected

If the web server has bugs → VPN is affected

Larger attack surface

So security depends heavily on:

Browser security

Web server security

Portal security

Types of SSL VPN
1. Portal VPN (Web Portal)
How it works:

User opens browser

Goes to VPN website

Logs in

Accesses internal resources via web

Limitations:

Only web-based resources

Very restricted

Good for:

Email

Web apps

Internal dashboards

2. SSL Tunnel VPN
How it works:

Browser creates secure tunnel

User can access many services

Similar to IPsec VPN

Requires:

Active content:

Java

JavaScript

ActiveX

Flash (old)

Good for:

Remote desktops

File servers

Internal applications

Security Implications of VPNs
Big mistake to avoid

Many companies think:

"We use VPN, so we are secure."

This is wrong.

VPN gives encryption, not total security.

Problem 1: Over-trusting the VPN
Site-to-Site VPN risk

If you connect your network to another company:

Their network becomes part of your network

Any problem on their side → affects you

If it’s your own branch:

Fine, same policies

If it’s a partner:

Very dangerous if unrestricted

Best practice:

Use firewall rules on the VPN gateway:

Allow only required services

Block everything else

Problem 2: Encryption blinds your security tools

Encryption:

Protects from attackers

But also hides traffic from:

IDS

IPS

Network monitoring tools

So:

You can’t inspect what you can’t see.

Malware can pass through the VPN tunnel invisible.

Problem 3: Trusted client problem (Remote users)

Client-to-site VPN:

User connects from home device

That device might:

Have malware

Be unpatched

Be compromised

Once connected:

That infected device is now inside your network.

Attackers can:

Move laterally

Access internal systems

This is called:
Pivoting.

Real-world example

Employee laptop:

Gets infected at home

Connects via VPN

Malware enters corporate network

No firewall stops it:

Because VPN is trusted.

Golden security rules for VPNs
1. Never trust VPN blindly

Always apply:

Firewall

Segmentation

Access control

2. Inspect VPN traffic where possible

Use:

Decryption zones

Endpoint security

Zero Trust models

3. Treat remote clients as untrusted

Especially:

BYOD devices

Use:

NAC (Network Access Control)

Endpoint checks

Limited access

On-The-Fly Encryption (Full Disk Encryption)

What does “Full Disk Encryption” really mean?

It means:

Data is always encrypted before it is saved to the hard disk.

How it works:

File is created in RAM (memory)

File is encrypted in RAM

Encrypted file is saved to disk

When needed:

File is loaded into RAM

Decrypted in RAM

User can use it

So:

Disk → always encrypted

RAM → decrypted only while in use

What does Full Disk Encryption protect?
Protects against:

Loss or theft

If laptop is stolen:

Attacker removes hard drive

All data is encrypted

Useless without key

What does it NOT protect against?

If:

Computer is ON

User is logged in

User is hacked

Then:

Attacker sees everything the user can see.

Because:

Decryption happens automatically

In real time

On behalf of the user

Encryption cannot help here.

The big lesson

Encryption is not magic.

It only protects against specific risks.

Full disk encryption:

Protects data at rest

Does NOT protect data in use

Simple analogy

Laptop stolen:

Full disk encryption = safe

Laptop unlocked:

Attacker = full access

GPG – Crypto Made Easy
What is GPG?

GPG (GNU Privacy Guard) is a tool for:

Encrypting files

Encrypting emails

Digitally signing messages

It works on:

Linux

Windows (Gpg4win)

macOS

What can GPG do?
1. Encrypt files

So only people with the key can read them.

2. Encrypt emails

So no one else can read your email.

3. Digitally sign emails

So receiver knows:

You sent it

It was not modified

How encrypted email works (simple)

Alice wants to send Bob a secret email.

Step 1

Alice generates a symmetric key.

Step 2

Alice encrypts the email with that key.

Step 3

Alice encrypts the key using Bob’s public key.

Step 4

Alice sends:

Encrypted email

Encrypted key

Establishing a Key and Choosing a Passphrase (GPG)
Step 1: Generate a Public/Private Key Pair

GPG lets you create:

A public key → can be shared with others

A private key → must stay secret

These keys are based on asymmetric cryptography.

Step 2: Choose the Algorithm and Key Size

When creating the keys, you need to select:

Which asymmetric algorithm to use (e.g., RSA, ECC)

The key length (longer = stronger, but slower)

Step 3: Protect the Private Key with a Passphrase

Your private key will be stored on your computer.
Anyone who gets this key can:

Sign messages as you

Decrypt messages meant for you

To protect it:

GPG encrypts the private key using a passphrase.

You must remember this passphrase to unlock your private key.

How to choose a strong passphrase

Length: Long is better (e.g., 12+ characters)

Complexity: Include letters, numbers, symbols

Memorable: Something only you can remember

Not guessable: Avoid obvious words or patterns

Think of it as a “lock” for your private key.

Key security is critical

Your private key represents you digitally.

Digital signatures made with your key are legally binding, like a physical signature.

Losing your key or passphrase is serious.

Step 4: Using GPG

GPG can encrypt or sign content easily.

Many graphical interfaces make it as simple as clicking a button.

You can also use command-line for automation.

What Is a Public Key Infrastructure (PKI)?

A Public Key Infrastructure (PKI) is a system that manages digital certificates.

It is responsible for:

Creating certificates

Maintaining certificates

Revoking or expiring certificates

What is a Certificate?

A certificate is a digital document that:

Contains a public key

Binds that public key to a real identity (person, website, company, server)

In simple words:

A certificate proves “this public key really belongs to this identity.”

Why PKI is important

Public/private keys are used everywhere:

Secure websites (HTTPS)

Encrypted email

Digital signatures

VPNs

Software updates

But how do you know a public key is real and not fake?

That’s the job of PKI.

PKI provides:

A way to verify public keys

A way to trust identities

A way to manage keys at large scale

Chain of Trust

PKI works using a hierarchy of trusted authorities, called:

Certificate Authorities (CAs)

These authorities form a chain of trust.

Example:

You trust a Root CA (like DigiCert or Let's Encrypt)

That CA signs another CA

That CA signs a website certificate

Therefore, you trust the website

Certificates = Digital Passports

Certificates are like:

Passports

Driver's licenses

National ID cards

Why do we trust a passport?
Because a government authority issued it.

Same idea digitally:
We trust certificates because a trusted CA verified the identity before issuing it.

How Certificates Prove They Are Real

Certificates use digital signatures.

A certificate is digitally signed by a CA.
This allows your system to:

Verify the certificate is authentic

Detect if it was modified

Trust the identity inside it

If an attacker could fake certificates, all security would collapse.

Certificates: The Easy Picture

A certificate is created by a trusted organization called a
Certificate Authority (CA).

The CA's digital signature is what creates the binding between:

The identity (Bob)

The public key (Bob's public key)

This signature is what makes the certificate trustworthy.

Step 1: Bob Requests a Certificate

Bob goes to a Certificate Authority (CA) and says:

“I want a certificate for my public key.”

Step 2: The CA Verifies Bob’s Identity

The CA must check two things:

Bob is really Bob (identity check)

Like showing passport or ID.

The public key really belongs to Bob

This proves Bob owns the private key too.

Step 3: Proving Key Ownership

The CA does this test:

CA encrypts a secret using Bob’s public key

CA gives the encrypted secret to Bob

Bob decrypts it using his private key

Bob sends the secret back

If Bob can decrypt it, the CA knows:

Bob owns the key pair

The public key is legitimate

Step 4: The CA Creates the Certificate

Now the CA:

Hashes Bob’s public key

Encrypts that hash using CA’s private key

This creates a digital signature

The certificate contains:

Bob’s identity

Bob’s public key

CA’s digital signature

Step 5: Alice Verifies Bob’s Certificate

When Alice receives Bob’s certificate:

She hashes Bob’s public key

She decrypts the CA signature using CA’s public key

She compares both hashes

If they match:

The certificate is authentic

The public key is real

Bob is trusted

The 5 Main Stages of a Certificate Life Cycle

1. Certificate Registration

This happens before a certificate is issued.

The user (for example, Bob):

Provides identity information (called a Distinguished Name – DN)

Submits a CSR (Certificate Signing Request)

Contains Bob’s identity

Contains Bob’s public key

The CA verifies:

Bob’s identity

That the public key really belongs to Bob

2. Certificate Creation (Certification)

After verification, the CA:

Creates the certificate

Adds:

Bob’s identity

Bob’s public key

Validity period

Digitally signs it with CA’s private key

Now Bob officially has a trusted certificate.

3. Certificate Distribution

The certificate must be available to others.

Public keys should:

Be easy to access

Be stored in:

Directories (like LDAP)

Certificate servers

Or shared directly

This allows Alice to find Bob’s certificate and trust him.

Key Storage (Very Important)
Public Keys

Safe to share

Must be public

Used for:

Encryption

Verifying signatures

Identity lookup

Private Keys (Critical)

Private keys must be protected at all costs:

Client private keys:

Protected with passphrases

Stored securely

Never shared

If stolen → attacker becomes the user.

Server private keys:

Used for HTTPS, TLS, VPN

If compromised → man-in-the-middle attacks possible

CA private keys:

Most important of all

If compromised → entire PKI collapses

All certificates become untrusted

4. Certificate Revocation

If a private key is compromised:

The certificate must be revoked

You cannot “fix” a private key.
Instead, you:

Revoke the public key

Which automatically invalidates the pair

This is done using a:

CRL (Certificate Revocation List)
A list of all revoked certificates

Systems check the CRL before trusting a certificate.

Reasons for Revocation

Certificates can be revoked when:

Private key is stolen

Employee leaves company

Job role changes

Identity changes (name, organization)

Security incident occurs

5. Certificate Expiration

Certificates should not last forever.

They have:

Start date

End date

After expiration:

Certificate is no longer trusted

New keys should be generated (best practice)

This is called key rotation

Certificate Revocation List (CRL)
What is a CRL?

A CRL (Certificate Revocation List) is:

A list of certificates that are no longer trusted.

When a certificate is revoked:

The CA adds its serial number to the CRL.

Anyone who wants to trust a certificate must check this list.

How CRL Works (Simple Example)

Alice wants to trust Bob's certificate.

Her computer:

Downloads the CA's CRL.

Searches for Bob's certificate serial number.

If found → certificate is revoked.

If not found → certificate is still valid.

Problems with CRLs
1. Inefficient

The client must download the entire list.

Even if checking only one certificate.

2. Network-Heavy

CRLs can be very large.

Consumes bandwidth and time.

3. Caching Risk (Big Problem)

To reduce traffic:

Browsers cache the CRL.

But this causes a dangerous gap:

Time	What happens
T1	Certificate revoked
T2	Client still using old cached CRL
T3	Client unknowingly trusts a revoked cert

This window is called a revocation gap.

Delta CRLs

A small improvement:

Only download changes since last CRL.

Still does not fix the core problem.

Online Certificate Status Protocol (OCSP)

OCSP is the modern replacement for CRLs.

How OCSP Works

Instead of downloading a big list:

Client sends:

Certificate serial number

OCSP server replies:

Good

Revoked

Unknown

This is a one-to-one lookup.

Much faster and more efficient.

OCSP Privacy Problem

With OCSP:

The CA can see:

Which websites you are visiting

Because your browser contacts the CA directly

This is a privacy leak.

OCSP Stapling (Best Solution)

To fix privacy:

The server does the OCSP check, not the client.

Process:

Web server asks CA: "Is my cert valid?"

CA replies with signed OCSP response.

Server staples this response to its certificate.

Browser verifies signature.

No need to contact CA.

Secure Web Traffic: HTTPS (SSL/TLS)

HTTPS = HTTP + Encryption
It protects data between your browser and a web server.

The Two Protocols
1. SSL (Secure Sockets Layer)

Created in 1994 (by Netscape)

Deprecated and insecure

Should never be used today

Still mentioned only for history

2. TLS (Transport Layer Security)

Modern replacement for SSL

What HTTPS actually uses today

Much stronger and more secure

When people say “SSL certificate”, they really mean TLS certificate.

What Does HTTPS Provide?

Both SSL/TLS are designed to give:

Security Service	Meaning
Confidentiality	Data is encrypted
Integrity	Data cannot be modified
Authentication	Server identity is verified
Key exchange	Secure session key creation

How HTTPS Actually Works (Step by Step)

1. Browser connects to server

User goes to:

https://example.com

2. Server sends its certificate

The certificate contains:

Server identity

Server public key

CA digital signature

3. Browser verifies certificate

Browser checks:

Is it signed by a trusted CA?

Is it expired?

Is domain name correct?

If valid → trust is established.

4. Session key is created

Browser uses server's public key to:

Securely negotiate a symmetric session key

This uses:

Asymmetric crypto → for trust

Symmetric crypto → for speed

5. Encrypted communication begins

All data is now encrypted using:

AES / ChaCha20 (fast symmetric crypto)

Why Both Asymmetric and Symmetric?

Crypto Type	Used for
Asymmetric (RSA/ECC)	Trust & key exchange
Symmetric (AES)	Actual data encryption

Asymmetric is slow → used once
Symmetric is fast → used for all traffic

Server Authentication (Default)

In normal HTTPS:

Browser verifies server

Server does not verify browser

This is called:

Unilateral trust

Client Authentication (Optional)

Servers can require:

Client certificates

Used in:

Corporate VPN portals

Banking systems

Internal company systems

This is:

Mutual TLS (mTLS)

Important Security Notes
SSL is DEAD

If you see:

SSLv2

SSLv3

That is broken security.

Only accept:

TLS 1.2

TLS 1.3

HTTPS Is Not Just for Web

TLS is also used for:

Secure email (SMTP, IMAP)

VPNs

APIs

Database connections

IoT devices

Secure Web Traffic: HTTPS – Step by Step

Whenever you visit a website using https://, this process happens automatically in milliseconds.

1. Client Hello

The browser says:

“Hello, here are the security methods I support.”

It sends:

Supported TLS versions

Supported cipher suites

A random number

2. Server Hello

The server replies:

“Here is what we will use.”

It sends:

Chosen cipher suite

Another random number

Server digital certificate (contains public key)

3. Client Validates Certificate

The browser checks:

Is the certificate signed by a trusted CA?

Is it expired?

Does the domain match?

If valid → trust is established.

4. Client Creates Session Key

The browser:

Generates a symmetric session key

Encrypts it using server’s public key

Sends it to the server

5. (Optional) Client Certificate

If mutual authentication is required:

Client sends its own certificate

Server validates it

This is used in:

Banking

Corporate systems

VPNs

6. Server Decrypts Session Key

Server uses:

Its private key

To decrypt the session key

Now both sides share the same secret key.

7. Client Ends Key Exchange

Client confirms:

“Ready to switch to encrypted mode.”

8. Server Ends Key Exchange

Server confirms:

“Ready.”

9. Encrypted Communication Begins

All data now uses:

Fast symmetric encryption (AES/ChaCha20)

No one else can read or modify the traffic.

Visual Summary
Browser → ClientHello
Server → ServerHello + Certificate
Browser → Validate cert
Browser → Encrypted session key
Server → Decrypt session key
Browser ↔ Server → Encrypted data

Problems with Public Key Infrastructure (PKI)

PKI is powerful, but difficult to design, deploy, and manage, especially at large scale.

1. Competing or Incomplete Standards

There is no single perfect PKI standard that all systems fully support.

This causes:

Compatibility problems

Different vendors using different implementations

Applications not always working well together

So even if you build a PKI, some software may not support it properly.

2. Certification of Certificate Authorities (CAs)

A PKI is only as strong as the CA that issues certificates.

Problems:

How trustworthy is the CA?

Are their policies strict enough?

Do they properly verify identities?

If a CA is compromised or careless → the entire PKI becomes untrustworthy.

3. Do It Yourself or Outsource?

Organizations must choose:

Build your own CA:

Full control

High cost

Requires expert staff

Use a third-party CA:

Easier and cheaper

Less control

Trust is external

Both options have security, financial, and operational risks.

4. User Education and Perception

Users are often the weakest link.

Problems include:

Users ignoring certificate warnings

Not understanding trust messages

Resistance to smart cards or extra login steps

If users don't understand PKI → they may bypass security.

5. Extensive Planning Required

PKI is not plug-and-play.

You must plan:

Certificate lifecycle (issue, renew, revoke)

Backup and recovery

Key management

Disaster scenarios

Poor planning = broken PKI.

6. Lack of Critical Mass

PKI is useless unless enough systems use it.

Challenges:

High upfront cost

No immediate benefit

Hard to justify investment

Organizations may abandon PKI before it becomes useful.

Real-World Summary

Problem	Meaning
Standards	Not all systems agree
CA trust	One weak CA breaks everything
Cost	Expensive to build
Complexity	Hard to manage
Users	Often misunderstand it
Adoption	Needs large deployment to work

One-Line Exam Answer

PKI is difficult because it requires trusted authorities, strict standards, high cost, careful planning, and educated users. A single weak component can compromise the entire system.

A firewall is one of the most important security tools in any network.
Its main job is to control what traffic is allowed in and out of a system or network.

The name comes from real life:
A firewall in a building stops fire from spreading.
A network firewall stops attacks from spreading into your network.

1. Preventive Technology

A firewall works as a preventive control.

It is basically:

A router (or system) with a set of filtering rules

These rules decide:

What traffic is allowed

What traffic is blocked

So attacks can be stopped before they reach your systems.

2. Reduces Risk

Firewalls protect systems from:

Hackers trying to exploit vulnerabilities

Malware connections

Unauthorized access

Even if a system has weaknesses, the firewall can:

Block the attacker from ever reaching it.

3. Increases Privacy

Firewalls make it harder for attackers to:

Scan your network

Discover internal systems

Collect information about your services

This is called reducing information gathering (reconnaissance).

Less information = fewer successful attacks.

4. Enforces Security Policies

Firewalls help enforce rules like:

Employees cannot access certain websites

Only specific services (like HTTPS) are allowed

Remote access only through VPN

So the firewall becomes:

A technical way to enforce company security policies.

Where Are Firewalls Used?

Firewalls are commonly placed:

1. At the network edge

Between:

The public Internet

The organization's internal network

This is the most important location.

2. On individual devices

Between:

A PC's network card (NIC)

The operating system

This is called a host-based firewall (like Windows Firewall).

How Can a Firewall Be Implemented?

Firewalls can be:

Type	Example
Hardware appliance	Cisco, Fortinet
Software on router	Router with ACL rules
Software on PC	Windows Firewall, iptables

Firewall Benefits vs. Shortcomings

Firewalls are very powerful security tools, but they are not perfect.
They provide many benefits, but they also have limitations.

Think of a firewall like an umbrella :

It blocks most of the rain

But some drops still get through

Benefits of Firewalls
1. Protect Systems from Attacks

Firewalls:

Block unauthorized access

Stop many hacking attempts

Protect both internal and external systems

2. Filter Communications

Firewalls can filter based on content, for example:

Block malicious websites

Block offensive content

Prevent sensitive data from leaving the company

3. Network Address Translation (NAT)

Firewalls can hide internal IP addresses.

This:

Improves privacy

Makes internal systems invisible to attackers

4. Encrypt Communications (VPN / IPsec)

Firewalls can create secure tunnels:

Encrypt traffic between offices

Secure remote workers

5. Logging and Monitoring

Firewalls keep logs of:

Allowed traffic

Blocked traffic

These logs help with:

Intrusion detection

Incident response

Digital forensics

6. Defense in Depth

Multiple firewalls can be used together.

Example:

Perimeter firewall

Internal firewall

Host firewall

This layered approach is called:

Defense in Depth

Shortcomings of Firewalls
1. Application Layer Attacks

Firewalls may not stop:

SQL injection

XSS attacks

Malicious files inside allowed traffic

If traffic looks “normal”, it may pass.

2. Encrypted Traffic Bypasses Inspection

Firewalls cannot easily inspect:

HTTPS

VPN tunnels

Extranet connections

Malware can hide inside encrypted traffic.

3. False Sense of Security

Organizations may think:

"We have a firewall, so we are safe."

So they neglect:

System patching

Strong passwords

Endpoint security

Encryption

This is very dangerous.

4. Firewall Is Not a Silver Bullet

A firewall:

Can be misconfigured

Can be bypassed

Can be attacked itself

It cannot replace:

Antivirus

IDS/IPS

User awareness

Secure system design

Firewall Default Rules

Firewalls use rules to decide whether to allow or block network traffic. But sometimes a packet doesn't match any specific rule. That's where the default rule comes in.

1. Default Deny (Recommended ✅)

Definition:

If a packet doesn't match any specific rule, it is blocked by default.

Also called: "deny all except explicitly allowed".

Benefits:

Protects against unknown attacks or vulnerabilities

Ensures only explicitly approved traffic is allowed

Security posture:

Very restrictive

Safer for sensitive networks

Example:

Firewall rules allow only HTTP (port 80) and HTTPS (port 443).

All other traffic is automatically blocked, even if no rule mentions it.

2. Default Allow (Not Recommended)

Definition:

If a packet doesn't match any specific rule, it is allowed by default.

Also called: "allow all except explicitly denied".

Drawbacks:

Very permissive

Increases risk from unknown attacks

Could allow malicious traffic to pass through

Example:

Firewall rules block only FTP (port 21).

All other traffic, even potentially dangerous, is allowed by default.

Anti-Spoofing

Definition:
Anti-spoofing is a network security control that prevents devices from sending packets with a source IP address that doesn't match the network behind the interface it's coming from.

Purpose:

Stops attackers from pretending to be a trusted device (IP spoofing).

Ensures traffic originates only from its legitimate network.

How It Works (Example with Interfaces)

Imagine three interfaces:

Interface	Network Behind It	Expected Source IP
A	192.168.33.0	192.168.33.x only
B	192.0.2.0	192.0.2.x only
C	Internet	Any source other than A or B networks

Rule: Traffic from the 192.168.33.0 network must come only from interface A.

Rule: Traffic from the 192.0.2.0 network must come only from interface B.

Any packets with mismatched source addresses are blocked.

Example of Spoofing

A device behind interface B sends a packet with a source IP of 192.168.33.5.

The firewall sees this as spoofed traffic because 192.168.33.0 is supposed to come only from interface A.

Action: Packet is blocked.

Key Takeaways

Anti-spoofing prevents IP forgery.

Helps enforce network boundaries.

Commonly implemented on routers and firewalls at network edges.

Types of Firewalls

Firewalls are security devices that control traffic between networks. There are three general types:

Packet Filtering Firewalls

Stateful Firewalls (Stateful Packet Filtering)

Proxy Firewalls (Application Gateway / NGFW)

1. Packet Filtering Firewall

How it works: Examines individual packets based on rules (source/destination IP, port, protocol).

Pros: Very fast, low resource usage.

Cons: Not very secure; does not track connections, so it can't detect sophisticated attacks.

Use case: Good as a first layer to filter obvious unwanted traffic.

2. Stateful Firewall

How it works: Tracks the state of active connections in a state table. Uses this information to allow or block packets.

Pros: More secure than packet filtering because it knows the context of connections.

Cons: Slower and requires more resources.

Use case: Provides better security for sessions and ongoing traffic.

3. Proxy Firewall (Application Gateway / NGFW)

How it works: Intermediary between the client and server. The two systems do not communicate directly. The firewall inspects the traffic at the application layer.

Pros: Highest security; can filter content, inspect for malware, and hide internal network structure.

Cons: Slowest due to deep inspection; higher resource use.

Use case: Critical for protecting sensitive resources and high-security networks.

Typical Deployment Strategy

Firewalls are often layered for efficiency and security:

Packet filtering up front – filters obvious noise quickly.

Stateful firewall behind it – handles remaining traffic with context awareness.

Proxy firewall – inspects a small subset of traffic deeply, minimizing performance impact.

Stateless Packet Filter

A stateless packet filter is the most basic type of firewall.

Key Characteristics
Advantages

Very fast ⚡

Low cost

Uses minimal system resources

Good for filtering obvious unwanted traffic ("noise")

Disadvantages

Minimal security

Easy to bypass

No awareness of sessions or connections

Does not inspect data content

How It Works

Stateless firewalls examine each packet individually, based only on:

Source IP

Destination IP

Port number

Protocol (TCP/UDP)

TCP flags (SYN, ACK, RST, etc.)

They do not remember previous packets.

The ACK Flag Problem

Stateless firewalls rely heavily on TCP flags:

If ACK = 1, the firewall assumes:

"This packet belongs to an existing connection."

But this is only an assumption, not a verification.

Why This Is Dangerous

An attacker can:

Craft packets with ACK flag set

Send them through the firewall

The firewall allows them because it thinks the session already exists

This allows attackers to:

Bypass rules

Map firewall policies

Discover open ports

This attack technique is called an:

ACK Scan

Example Attack (ACK Scan)

Attacker sends TCP packets with ACK flag set to different ports.

Firewall lets them pass.

Target host replies with:

RST/ACK → Port is open/unfiltered

No response → Port is filtered

Attacker learns the firewall configuration.

Why Stateless Firewalls Still Exist

Even with weaknesses, they are still useful:

As a first layer in a defense-in-depth strategy

To quickly drop:

Bogus IP ranges

Unwanted protocols

Simple scanning traffic

They handle the easy work so stronger firewalls can focus on real threats.

Stateful Firewalls

A stateful firewall is a more advanced type of firewall that keeps track of active connections and uses that information to make smarter decisions about network traffic.

Key Features

Maintains state

Tracks source IP/port and destination IP/port for each connection.

Keeps a state table to know the current status of each connection.

Tracks connection progress

For TCP connections, it monitors the 3-way handshake: SYN → SYN-ACK → ACK

Example states:

SYN_SENT – First packet sent to initiate connection

SYN_RECV – Response received

ESTABLISHED – Connection fully established

FIN_WAIT1 / LAST_ACK / FIN_WAIT2 – Connection closing stages

CLOSED – No active connection

Smarter packet filtering

Each incoming packet is checked in context of its connection.

Can detect malformed packets or packets that don't belong to an active session.

Not vulnerable to ACK scan attacks like stateless firewalls.

Handling non-TCP protocols

UDP and ICMP don't have connection flags, so stateful firewalls use timeouts to remove inactive sessions.

ICMP error packets (e.g., "Port Unreachable") are tricky; firewalls may need to accept them blindly

What is an IDS (Intrusion Detection System)?

An IDS is like a security alarm for your computer or network.

Simple definition:

An IDS watches what is happening on a system or network and warns you if something looks like an attack.

Think of it like this

Imagine your house has:

Cameras

Motion sensors

An alarm system

The alarm:

Does not stop the thief

But alerts you that something suspicious is happening

IDS works the same way.

What does an IDS do?

An IDS:

Monitors network or system activity

Looks for suspicious or malicious behavior

Sends alerts when it finds something dangerous

What does an IDS NOT do?

Important:

IDS does NOT block attacks

IDS does NOT protect by itself

It only:

Detects

Reports

Alerts humans

Who reacts to IDS alerts?

A person called an Incident Handler:

Receives the alert

Analyzes what happened

Decides what action to take (block IP, shut down system, investigate, etc.)

Is IDS enough for security?

No ✕

IDS should be used with other tools, like:

Firewall

Antivirus

Patch management

Vulnerability scanners

This is called Defense in Depth (multiple layers of security).

What an IDS is NOT

Even though IDS is useful, many people misunderstand it.
So let's make it simple.

1. IDS is NOT a replacement for other security tools ❌

IDS cannot replace:

Firewalls

Strong security policies

System hardening

Regular updates (patching)

Antivirus

Easy example:

You can't remove:

The door lock

The walls

The police

And only keep the alarm.

IDS is just one layer, not the whole security.

2. IDS is NOT low-maintenance ❌

IDS:

Needs constant monitoring

Needs people to analyze alerts

Needs tuning and updates

It's not:

"Install and forget"

It’s more like:

“Install and babysit 24/7”

3. IDS is NOT cheap ❌

Even if the software is free:

You need servers

You need storage

You need trained analysts

You need time

The real cost is:

People + Time + Management

4. IDS is NOT a silver bullet ❌

“Silver bullet” means:

One tool that solves all problems

IDS is not magic.
It won’t:

Automatically secure your company

Make bad policies good

Replace human thinking

Very simple comparison:

Tool	What it does
Firewall	Blocks traffic
Antivirus	Removes malware
IDS	ينبه فقط (just alerts)

The biggest mistake with IDS

Buying IDS and thinking:

“Now we are secure”

But in reality:

No team to monitor

No plan to respond

No policies

So alerts are ignored = useless system

What are IDS Alerts?

When an IDS sees any activity, it must decide:

Is this important (attack)? Or normal?

This important activity is called:
EOI – Event Of Interest

يعني: مراقبته نريد مهم حدث.

The 4 types of IDS results

Every IDS decision falls into one of these four:

1. True Positive ✅ (حالة أفضل)

Meaning:
Attack happened ➜ IDS detected it correctly.

Example:

Hacker tries SQL Injection

IDS raises alert

✔ Correct alert.

2. False Positive ❌ (كاذب إنذار)

Meaning:
No attack ➜ IDS thinks it's an attack.

Example:

Normal user downloads big file

IDS thinks it's data exfiltration

❌ Wasted time for analyst.

3. True Negative ✅ (حالة طبيعية)

Meaning:
No attack ➜ IDS says no attack.

Example:

Normal browsing

IDS stays silent

✔ Perfect behavior.

4. False Negative (حالة أسوأ)

Meaning:
Attack happened ➜ IDS misses it.

Example:

Malware communicates with server

IDS says nothing

This is very dangerous.

Super easy table

Reality	IDS says	Result
Attack	Attack	True Positive
Normal	Attack	False Positive
Normal	Normal	True Negative
Attack	Normal	False Negative

Which is the worst?
Worst = False Negative **!**

Because:

Attack happens

No alert

Analyst never knows

This is how hackers stay hidden.

Which is annoying?
Most annoying = False Positive

Because:

Too many alerts

Analysts get tired

Real attacks might be ignored

The big challenge

IDS must balance between:

Not too many false positives

Not too many false negatives

Too sensitive → noise
Not sensitive → blind

What is NIDS?

NIDS = Network IDS

It is an IDS that:

Watches the network traffic instead of a single computer.

How does NIDS work?

NIDS is deployed as a passive sniffer.

يعني:

It فقط "يتفرج" على الترافيك

Does NOT block anything

Does NOT change packets

Just:

Sniff → Analyze → Alert

Where is NIDS placed?

NIDS is placed at important network points, like:

Core switch

Gateway

Data center link

Using:

SPAN / Mirror port (copy of traffic)

Or Network TAP (physical splitter)

So it can see everything passing.

What does NIDS do?

It:

Captures network packets

Looks for Events of Interest (EOI)

Generates alerts if it sees attacks

How does NIDS detect attacks?

NIDS uses 3 main techniques:

1. Signature-based detection

Like antivirus.

Compares traffic with known attack patterns.

Example:

Known exploit string

If found → alert

2. Anomaly-based detection

Looks for strange behavior.

Example:

User normally sends 1MB

Suddenly sends 10GB

Suspicious → alert

3. Protocol / Application analysis

Understands how protocols should behave.

Example:

HTTP request that breaks rules

DNS packet with invalid format

Attack → alert

Software vs Hardware NIDS

NIDS can be:

Software on a server

Or dedicated device (appliance)

Both do same job:

Analyze traffic + raise alerts

What is Signature Analysis?

Signature Analysis means:

The IDS looks for known attack patterns in the traffic.

Exactly like:

Antivirus scans files using virus signatures.

How it works (simple steps):

IDS has a list of signatures (rules)

Each signature = pattern of a known attack

IDS checks every packet

If pattern matches → ALERT

What is a signature?

A signature can be:

A specific string

A known exploit pattern

A file hash

A protocol behavior

Example:

String: "cmd.exe /c"

SQLi: ' OR 1=1 --

Why rules are important?

Each rule tells IDS:

"If you see THIS → it's an attack"

Bad rules → bad results:

Wrong rule → False Positive

Missing rule → False Negative

What are IDS Rules?

Rules are instructions that tell IDS:

"If you see this pattern → raise alert"

They are like:

IDS للـ المرور قوانين

Why rules are important?

Because:

Default rules are not enough

Every network is different

You may want to detect custom attacks

So IDS must have:

Flexible rule language

بسهولة قواعد وتعدل تضيف تقدر يعني.

What can rules check?

IDS rules can look at almost anything inside a packet:

1. Protocol

Which protocol?

TCP

UDP

ICMP

IP

Example:

Alert on ICMP floods

2. Address (IP)

Source or destination IP.

Example:

Alert if external IP accesses internal DB server

3. Port

Which port is used?

Example:

Alert on traffic to port 4444 (common malware port)

4. Payload (content)

Actual data inside packet.

Example:

Detect buffer overflow using repeated NOP (0x90)

5. String matching

Search for text.

Example:

"Login incorrect"

"cmd.exe"

"SELECT * FROM"

6. Traffic flow

Direction of traffic:

Outside → Inside

Inside → Outside

Helps reduce false positives.

7. Protocol flags

Check header flags.

Example:

TCP flags: SYN, FIN, RST

IP reserved bits

Example:

SYN + FIN together = suspicious

What is Anomaly Analysis?

Anomaly Analysis means:

The IDS learns what is normal behavior,
then alerts when something abnormal happens.

Step 1: Build a Baseline

First, IDS must know:

What does “normal” look like?

This is called Baseline:

Normal traffic size

Normal protocols

Normal users behavior

Normal time patterns

Step 2: Monitor traffic

IDS watches traffic continuously.

Step 3: Detect anomalies

If something is:

Too big

Too fast

Too different

Unexpected

→ IDS raises alert.

Simple example:

Normal:

Web server receives 100 requests/min

Suddenly:

50,000 requests/min

→ Anomaly → Possible DDoS

Why anomaly analysis is powerful?

Because it can detect:

Zero-day attacks

What is Application/Protocol Analysis?

It means:

The IDS fully understands how a protocol should work,
and flags anything that breaks the rules.

How it works (simple steps):

IDS studies the protocol deeply
(HTTP, DNS, FTP, SMTP, etc.)

Builds internal logic:

How input should look

How processing should happen

How output should behave

Monitors real traffic

If traffic does NOT follow protocol logic → ALERT

Example:

HTTP normal:

GET /index.html HTTP/1.1

Weird HTTP:

GET \x90\x90\x90\x90 HTTP/1.1

Breaks protocol rules → alert.

Why is this powerful?

Because it can detect:

Known attacks

Unknown attacks

Zero-days

As long as they:

Abuse the protocol logic

What it cannot detect?

If attack:

Uses protocol correctly

But exploits bad configuration

IDS may miss it.

Example:

Admin panel exposed publicly

No protocol violation

Still dangerous

Why is it difficult?
1. Protocol standards are messy

Not all protocols are clear or strict.

2. Different implementations

Same protocol, different behavior:

Cisco vs Microsoft vs Linux

3. Protocols change

New versions → new rules → IDS must update.

What is Packet Inspection?

Packet inspection means:

How deeply the IDS looks inside the packet.

There are two levels:

Shallow (سطحي)

Deep (عميق)

Shallow Packet Inspection (SPI)
What it does:

Looks only at headers

Maybe a little payload

Fixed positions

What it checks:

Source IP

Destination IP

Port numbers

TCP flags

Packet size

Advantages:

Very fast ⚡

Works at wire speed

Low CPU usage

Disadvantages:

Not very accurate

Misses complex attacks

Deep Packet Inspection (DPI)
What it does:

Looks at everything

Headers + full payload

Even variable-length fields

Tracks session state

What it checks:

Actual data content

Application logic

Protocol structure

Advantages:

Very accurate

Detects advanced attacks

Needed for anomaly & protocol analysis

Disadvantages:

Slow

Heavy CPU

Hard to scale

In real life?

Modern IDS uses:

Both together

Shallow:

Filters traffic quickly

Deep:

Inspects suspicious traffic deeply

Why is normalization important?

Because attackers try to confuse the IDS.

They use:

Encoding

Obfuscation

Fragmentation

Weird formatting

Goal:

Make attack look different
But still work on the victim

Example (simple)

Normal attack:

cmd.exe /c net user hacker

Obfuscated:

c%6Dd.exe /c n%65t us%65r h%61ck%65r

Same attack, different shape

Without normalization:

IDS may miss it

With normalization:

IDS decodes it back to normal

Then detects attack

What does IDS normalize?

IDS can:

Decode URLs

Reassemble fragments

Remove padding

Standardize encodings

So all traffic becomes:

One clean format

Why analysts love it?

Because it gives:

Consistent data

Reliable rules

Less evasion

Better detection

Data normalization is like:

Cleaning dirty text before spell-check.

If text:

h@ck3r

Normalizer:

hacker

Now detector works perfectly.

What are the advantages of NIDS?

NIDS gives organizations visibility and control over network traffic beyond just prevention.

1. Scalability

Can monitor large networks

Works on multiple segments without slowing down

2. Insight into network traffic

Shows who is talking to whom

Breaks down traffic by protocol and application

Helps identify misuse (e.g., too much P2P traffic)

3. Detect network problems

Misconfigured devices

Inefficient traffic patterns

Early warning for operational issues

4. Swift incident response

Alerts admins to attacks quickly

Helps stop damage faster

Shows probes, reconnaissance, and exploits

5. Auditing & compliance ✅

Checks if firewalls and policies are working

Generates alerts for unauthorized traffic

Helps track policy violations or rule changes

6. Additional flexibility in security

Can create custom rules for sensitive info

Example: Honeytokens

Label secret data with unique keywords

NIDS alerts if the data leaves network

Helps identify leaks and unauthorized sharing

7. Detect attacks more aggressively ⚡

Unlike firewalls that block, NIDS monitors deeper

Can identify attacks that bypass preventive measures

Other practical uses

Traffic analysis for network expansion planning

Detect compromised systems quickly

Provides data for forensic investigation

Challenges to remember

Network topology and access issues

Encrypted traffic is hard to analyze

Big networks = lots of alerts → need good management

High cost for proper deployment and maintenance

What is Snort Rule Flexibility?

Snort allows admins to write custom rules for nearly any traffic pattern.

Think of it as giving the IDS superpowers to detect anything you can imagine.

Key points
1. Powerful rule language

Can detect almost any Event of Interest (EOI)

Includes:

Worms

Exploits

Viruses

Unauthorized access

Honeytokens

2. Quick updates ⏱️

Snort community shares new rules fast

New worms, vulnerabilities → rules published quickly

Organizations can protect themselves sooner

3. Supports custom detection

Analysts can create rules for:

Sensitive data (honeytokens)

Specific internal applications

Any unique network activity

4. Trade-offs

Fast rule development → may cause false positives/negatives

Slow, methodical rule development → may miss new attacks

Easy example

Rule to detect suspicious SQL command:

alert tcp any any -> 192.168.1.5 1433 (msg:"Possible SQL Injection"; content:"xp_cmdshell"; sid:1000001;)

Alerts when xp_cmdshell is used in SQL traffic

Why this is important

Flexibility → Detect new threats fast

Customization → Fit specific network needs

Community → Always up-to-date with new attacks

What are Snort Rules?

Snort rules tell the IDS:

“Look for this traffic → Do something”

Snort comes with hundreds of ready-made rules, but you can write custom ones.

What you can do with a rule

When writing a rule, you have three main actions:

Option	What it does
Pass	Ignore the packet completely
Log	Save the packet info to a log, but do nothing else
Alert	Send an alert (e.g., syslog or file) and log the packet

How to write a simple rule

Example: Detect xp_cmdshell in SQL traffic to server 192.168.1.5:

alert tcp any any -> 192.168.1.5 1433 (msg:"Possible SQL Injection"; content:"xp_cmdshell"; sid:1000001;)

alert → action (send alert)

tcp any any → 192.168.1.5 1433 → source/dest IP & port

content:"xp_cmdshell" → pattern to match

msg: → message shown in alert

sid: → unique rule ID

Why this is useful

Can filter specific traffic

Can log suspicious packets

Can alert immediately on intrusion signs

Train your staff

IDS analysis is a high-skill job

Analysts must know how to:

Differentiate true positives from false positives

Respond to alerts properly

Poorly trained staff → overwhelmed → IDS underused

2. Use a management console

Combines data from multiple IDS sensors

Helps correlate alerts

Reduces false positives problem

Can use passive sniffers to get extra visibility

3. Consider SIEM solutions

SIEM = Security Information and Event Management

Combines:

IDS alerts

Syslog messages

Windows event logs

Other security sources

Gives a single view of network risk

4. Prepare for incident response

Have policies in place:

What happens if an internal user attacks via VPN?

Who investigates compromised hosts?

IDS provides the information needed for incident response

Exercise your incident response team regularly

5. Perform ROI analysis

Decide: in-house or outsourced IDS management

Managed security services can be expensive, but may save long-term costs

Even with outsourcing, staff must be trained for incident handling

1. Reduction of False Positives

Problem: NIDS often alerts on traffic that isn't really malicious.

Solution: Target OS identification (passive fingerprinting)

IDS guesses the operating system of the host

Uses network traits like:

TCP window size

IP TTL

TCP flags order

Only raises alerts relevant to that OS

Example: Don't alert Windows exploits against a UNIX server

Benefit: Less useless alerts → analyst less overwhelmed

2. Integrated Vulnerability Assessment

Goal: Alert only on attacks that can actually succeed

IDS keeps track of vulnerabilities on hosts

Alerts only if the system is truly vulnerable

Can be:

Passive: IDS observes existing network data

Active: IDS or tool scans hosts to check vulnerabilities

Benefit: Alerts are more accurate and prioritized

3. NIDS Integration in Networking Devices

NIDS now built into:

Routers

Switches

Firewalls

Helps simplify deployment and reduce latency

4. IDS for Wireless Networks

Wireless networks have different traffic patterns

New NIDS versions can monitor Wi-Fi and detect attacks in wireless environments

1. Passive Vulnerability Analysis

The IDS observes a host's traffic to detect vulnerable services without actively scanning.

If the traffic shows signs that a service is vulnerable, the IDS marks it in its database.

Advantage: No disruption to production systems.

Limitation: May miss some vulnerabilities because it can't see everything.

2. Challenges in Modern Networks

Switched networks make traffic monitoring harder.

High-speed networks can overwhelm IDS.

Solution: Use modular hardware ("IDS blades") that can plug into switches and monitor traffic without slowing the network.

Custom hardware buffers packets so IDS can analyze traffic even when traffic is very fast.

3. Wireless IDS (WIDS)

Wireless networks have unique threats not seen in wired networks.

WIDS monitors wireless traffic specifically, instead of only upper-layer protocols like IP.

Can detect rogue Wi-Fi devices in unauthorized areas.

Uses:

Signature analysis

Anomaly analysis

Purpose: Detect attacks that bypass standard wireless security

What is IPS?

IPS is like IDS on steroids.

Instead of just alerting when an attack happens (like IDS), IPS stops the attack automatically.

IPS protects both networks and hosts.

2. Types of IPS

Type	Where it works	What it does
NIPS (Network IPS)	Network	Monitors traffic like a NIDS and blocks attacks inline
HIPS (Host IPS)	Individual host	Protects a single computer, stopping attacks on OS or apps

3. How IPS Works

Early IPS: Active IDS that sent TCP resets to attacker hosts.

Modern IPS:

Automatically stops attacks in real-time

Works inline (all traffic passes through it)

Reduces false positives and system impact

Can stop known and unknown attacks

4. IPS Advantages

Stops hackers, malware, worms, and viruses before they succeed

Adds an extra layer of defense on top of firewalls and IDS

Improves security for both networks and individual systems

Easy Analogy

IDS = “Alarm system” → tells you when someone is breaking in

IPS = “Alarm system with security doors” → stops them immediately

What an IPS is not:

Not a replacement for other security measures

An IPS can help stop attacks, but it cannot replace firewalls, patching, strong security policies, or making systems stronger. You still need all of those. Think of IPS as one layer in a multi-layer security plan.

Not a “set-and-forget” tool

You can’t just install it and forget it. IPS needs constant monitoring and maintenance to stay effective.

Not cheap

Enterprise IPS solutions can be expensive. You have to consider costs for hardware, software, and ongoing support.

Not a silver bullet

It won’t stop every attack or solve all security problems. No single tool can do that.

Why you still use IPS:

Even though it has limits, an IPS is useful because it gives you extra time. For example:

If a system has a vulnerability, IPS can help block attacks temporarily until your team has time to patch the system properly.

This allows you to plan maintenance and apply multiple patches together safely.

Key takeaway: IPS is an important layer, but it works best as part of a bigger, multi-layered security strategy (defense-in-depth).

How NIPS Works (Easy Explanation)

NIPS (Network Intrusion Prevention System) is a security system that monitors network traffic and blocks attacks automatically.

Think of NIPS like a security guard for your network.

Where is NIPS placed?

NIPS is usually placed at the edge (perimeter) of the network, near the internet connection.

There are two common places:

1 Between ISP Router and Firewall

(Outside the firewall)

This means:

Internet → ISP Router → NIPS → Firewall → Servers (DMZ)

Why here?

NIPS stops attacks before they even reach the firewall

Protects public servers (like web and mail servers)

Reduces load on the firewall

So it acts as the first line of defense.

2 Behind the Firewall

(Inside the network)

This means:

Internet → Firewall → NIPS → Internal Network

Why here?

Protects the internal network from:

Hackers who passed the firewall

VPN users

Infected devices inside the company

So it acts as a second line of defense.

In short (Very simple)

NIPS watches all network traffic

If it sees an attack → it blocks it immediately

It is placed:

Before firewall → to protect servers

After firewall → to protect internal users

How NIPS Hardware Works (Easy Explanation)

Why NIPS uses special hardware (ASIC)

NIPS devices use something called ASIC (Application-Specific Integrated Circuit).

This means:

The processor is built for one job only

Not like your PC CPU (which does many things)

It only focuses on analyzing network traffic

Because it does only one job, it is:

Faster

More efficient

More reliable

2- Speed Requirement

NIPS must handle very high network speeds, like:

Gigabit Ethernet (1 Gbps or more)

Because:

It sits in the middle of traffic

All packets must pass through it

If it's slow → your whole network becomes slow

3- Low Latency (No noticeable delay)

Even though NIPS checks every packet:

It must add only a few milliseconds of delay

Users should not feel any slowdown

So NIPS must be:

Fast AND smart at the same time

4- Security of the NIPS itself

NIPS must be very hard to hack, because:

If an attacker hacks the NIPS:

They can spy on all traffic

They can modify traffic

This becomes a Man-in-the-Middle attack

So NIPS is protected by:

No IP or MAC on data interfaces

Hardened OS

Secure management access only

5- How NIPS processes traffic efficiently

NIPS uses a smart filtering method:

Step 1: Simple checks (fast)

Example:

If packet has no data → skip it

No need to waste CPU.

Step 2: Medium checks

Looks at:

Headers (IP, TCP, UDP)

Session state

Step 3: Deep checks (slow but accurate)

Only for suspicious packets:

Payload inspection

String matching

Application analysis

Regular expressions

This method is called:

Multiresolution Filtering

Meaning:

Easy tests first

Hard tests only when needed

So NIPS saves resources and stays fast.

NIPS Challenges (Easy Explanation)

NIPS is powerful, but it has big challenges because it works inline (in the middle of traffic).

1-False Positives (Biggest Problem)
What is a false positive?

When NIPS thinks:

“This is an attack”
But in reality:
“This is normal traffic”

So NIPS blocks real users ❌

This causes:

Website not working

Emails not delivered

Users disconnected

Which is basically:

Denial of Service (DoS) created by your own security system

That’s why:

Organizations cannot afford false positives with NIPS.

2- Throughput Problem (Speed)

NIPS must:

Analyze traffic

Decide block or allow

Do it in real time

If traffic is too high:

NIPS can't keep up

Packets get dropped

Network becomes slow or unusable

Again → DoS risk

3 Latency Problem (Delay)

Every packet must wait for NIPS to:

Inspect it

Make a decision

This adds delay (latency).

If NIPS is slow:

Applications feel laggy

Video calls stutter

Websites load slowly

4 Smaller Rule Base

NIPS uses fewer rules than IDS.

Why?
Because:

Every rule costs CPU

More rules = more delay

More rules = more false positives

So vendors prefer:

Fewer, safer rules

Lower risk of blocking legit traffic

5 More False Negatives than IDS

Because NIPS uses fewer rules:

Some attacks are missed

This is called false negative

So:

IDS is better at detection

NIPS is safer for blocking

Developments in NIPS (Easy Explanation)

Over time, NIPS systems have become smarter, faster, and more useful.

1 Better Speed & Performance

Modern NIPS:

Handle more traffic

Add less delay

Can analyze traffic almost in real time

So now:

Security without slowing down the network.

2 Automated Updates (Good and Bad)

NIPS can:

Automatically download new attack signatures

Protect against new threats quickly

But the risk:

Bad signature → blocks real traffic

Causes false positives → DoS

So:

Auto-updates are powerful, but must be controlled by policy.

3 Environmental Anomaly Detection

NIPS learns:

What is “normal” for YOUR network

Example:

10,000 web requests/minute

Normal for a public website

Abnormal for a finance department

So NIPS can:

Detect strange behavior

Even if it’s not a known attack

4 Passive & Broad Analysis

Because NIPS sees all connections:

It can detect patterns across:

Many users

Many servers

Entire enterprise

This gives it a big-picture view.

5 Protocol Scrubbing (Traffic Cleaning)

NIPS can remove garbage traffic.

Example:

A server keeps sending packets to a dead computer

NIPS:

Drops those packets

Or fakes a reply to stop the server

Result:

Less useless traffic → better performance.

6 Rate Limiting & Policy Enforcement

NIPS can act like traffic police

It can:

Limit bandwidth per user/app

Prioritize important traffic

Stop one system from using all internet

Example:

You limit YouTube

Give priority to VPN and email

Passive Analysis (Easy Explanation)

Passive Analysis means:

NIPS learns about your network by watching traffic, not by scanning or attacking systems.

So it becomes smarter about what is normal in your environment.

1 Learning the Network (Network Learning Mode)

NIPS can run in “network learning mode”.

In this mode it:

Discovers devices

Learns:

Network structure

IP ranges

Routers

Servers

Normal paths

This creates a map of your network.

2 Identifying Operating Systems & Vulnerabilities

By just observing traffic, NIPS can guess:

This host runs Windows

This server runs Linux

This device has certain vulnerabilities

So later, when it sees an attack:

It asks:
“Is this attack even possible on this system?”

If not:

It ignores it → reduces false positives.

3 Smarter Attack Classification

Example:

Attack is for Linux

Target is Windows

NIPS knows:

This attack cannot work → drop alert or ignore.

So NIPS becomes context-aware.

4 Detecting Spoofed Internal Attacks

NIPS can detect fake internal traffic using:

IP ranges

If packet claims:

Source = internal IP
But:

IP not in learned network

→ It's spoofed → drop it.

TTL (Time To Live)

TTL shows:

How many hops a packet passed

If suddenly:

TTL is very different for same IP

→ Packet may be fake.

Why Passive Analysis is Important

Without it:

NIPS blocks too much

Many false positives

With it:

NIPS understands your environment

Makes better decisions

More accurate protection

Core Components of Endpoint Security (Easy Explanation)

Endpoint means:

Any device connected to the network
(laptop, PC, server, mobile, VM)

The main goal of endpoint security is not only to prevent attacks, but to:

Reduce the damage when attacks happen.

Because:

Attacks WILL happen

The question is: How bad will it be?

1-Limit the Attack Surface

Attack surface = all possible ways an attacker can enter

We reduce it by:

2- Patching

Fix OS and software bugs

Close known vulnerabilities

3-Updating services

Old software = easy target

4- Turning off unused services

If you don't need it → disable it

No service = no attack path

5-Controlling access

Strong passwords

Least privilege

Users only get what they need

2- Think About Pivot Points

When one system is hacked:

Attacker uses it to attack others

This is called a pivot

So endpoint security must:

Protect not just the device, but the whole network.

3- Strong Foundation (Most Important Part)

Security starts with basics:

4-Asset Inventory

Know:

What devices exist?

Who owns them?

What OS and software?

If you don’t know what you have → you can’t protect it.

5- Configuration Management

Ensure:

Secure settings

Same standards everywhere

No weak configs

6-Change Control

Track:

What changed?

Who changed it?

Why?

Uncontrolled changes = hidden vulnerabilities.

Prevention vs Detection

Prevention is ideal, but detection is a must.

Even with:

Antivirus

Firewall

EDR

Some attacks will succeed.

So:

You must detect fast

Limit damage

Respond quickly

What is a Baseline? (Easy Explanation)

A baseline is:

A record of how a system normally looks and behaves.

It is your “normal state” reference.

Why Baselines are Important

If you don’t know what is normal:

You can’t know what is wrong.

With a baseline:

You can quickly detect:

Attacks

Misconfigurations

System failures

How a Baseline is Created

You create a baseline by:

Running system commands

Saving their output (logs)

This becomes your normal snapshot

Examples of things you log:

Running services

Open ports

Users

Processes

Network connections

CPU & memory usage

How Baselines Are Used

Later, you:

Run the same commands again

Compare results with old logs

If something changed:

New service started?

New user added?

New port open?

Then:

You investigate.

Example (Very Simple)
Normal (Baseline):
Ports open: 80, 443
Services: nginx, ssh

Later:
Ports open: 80, 443, 4444
Services: nginx, ssh, unknown_service

That’s a red flag

Informal vs Formal Baselines
Informal baseline:

Admin just "feels" something is wrong

Based on experience

Formal baseline:

Real logs and comparisons

Much more accurate

Works even if admin is new

Establishing a Baseline (Easy Explanation)

To establish a baseline means:

Creating a clear picture of what “normal” looks like for your systems and network.

This “normal picture” is later used to detect:

Attacks

Misuse

Failures

Strange behavior

What Should Be in a Baseline?

When building a baseline, you must define:

1- Network Traffic

What protocols are used? (HTTP, HTTPS, SSH, DNS)

Which ports are open?

When are they used?

2- Amount of Traffic

How much traffic is normal:

Morning?

Night?

Weekends?

Example:

High traffic at 9 AM → normal

High traffic at 3 AM → suspicious

3- Logs

What types of logs exist?

How many logs per day?

Example:

1,000 logs/day → normal

50,000 logs/day → something wrong

4- Resource Usage

CPU usage

RAM usage

Disk activity

Systems usually follow patterns.

5-User Access Behavior

Who logs in?

From where?

At what times?

For how long?

Example:

Admin logs in at 9–5 → normal

Admin logs in at 3 AM from another country → suspicious

Baseline for Servers

You should also take a snapshot of servers:

Current configuration

Installed software

Important files

Then:

Hash the files

Monitor changes

If file hash changes:

Something was modified → investigate.

When Should You Create a Baseline?

Best time:

During a new network build

Reality:

Usually done on a live system

So:

Start logging immediately

Collect data over time

Look for stable patterns

Why More Data Is Better

The more you log:

The better your baseline

The easier to detect anomalies

Because:

You compare “today” with “normal history”.

Detecting Anomalies (Easy Explanation)

An anomaly is:

Anything that is different from normal behavior.

In security:

An anomaly usually means:

Attack

Misconfiguration

Malware

Insider activity

Key Idea (Very Important)

There is no such thing as an invisible attacker.

Even the best attacker will:

Change files

Start processes

Open connections

Modify configs

Generate logs

So:

If something changes → it can be detected.

How We Detect Anomalies
1- Use Baselines

First:

Know what is normal

Then:

Compare current state with baseline

Differences = anomalies.

2- Use Tools (Not Manual Work)

Checking one system manually is hard.

So we use:

File integrity checkers

Log collectors

Monitoring systems

These tools:

Automatically compare

Alert on changes

3- Visualize the Data

Graphs and dashboards make it easy:

Instead of:

Reading millions of logs

You see:

Spikes

Drops

New patterns

Outliers become obvious

Example (Simple)

Normal:

CPU: 20–40%

Logins: 9 AM – 5 PM

Anomaly:

CPU: 95% at midnight

Login at 3 AM from new IP

That’s suspicious.

Centralized Detection is Better

Best setup:

All systems send logs to one place

One dashboard

One timeline

This lets you:

See patterns across the whole network

Detect attacks faster

Looking for Anomalies (Easy Explanation)

Finding anomalies is not just about big changes in numbers.

It’s also about unexpected patterns or behavior.

1- Volume-Based Anomalies

Sometimes anomalies are easy to spot:

Example:

Normal: 100 logins/day

Now: 5,000 logins/day

This is a big spike → clearly suspicious

Investigate immediately

2- Behavior-Based Anomalies

Not all anomalies are obvious in numbers.

Sometimes the problem is in behavior patterns across multiple factors:

Example:

Charlie usually logs in from New York at 9:30 AM

15 minutes later, his account logs in from Germany

Analysis:

Short time between logins

Impossible location jump

Conclusion:

Highly likely suspicious activity

3- Correlation is Key

You find anomalies by correlating multiple events:

Time of access

Location/IP address

Type of activity

Resource usage

If two or more factors deviate at the same time → anomaly detected

4-Why This Matters

Attackers often try to blend in

Small deviations over multiple metrics can reveal stealthy attacks

Using just volume alone may miss these subtle attacks

Antivirus Software (Easy Explanation)

Antivirus (AV) software is:

Software that checks computers for viruses, worms, or other malicious programs and stops them from causing damage.

It is also called endpoint security software because it protects individual devices (endpoints) like:

Laptops

Desktops

Servers

What Antivirus Does

AV software typically:

Scans files

Checks your documents, programs, downloads

Checks emails & attachments

Makes sure nothing malicious comes through

Monitors web content

Prevents malware from websites

Looks for indicators of compromise (IoCs)

Signs that a system might be infected

History / Example

In the 1990s, viruses were the main threat

Example viruses:

Melissa

I Love You

Viruses attach themselves to programs

When you run the program → your system gets infected

Antivirus software was developed to detect and stop these threats.

How It Protects

Prevents infected files from running

Stops malware before it spreads

Often comes pre-installed on computers

Types of Antivirus Software (Easy Explanation)

Antivirus (AV) software today is not just for viruses anymore.

It is now often called anti-malware because it protects against many kinds of malicious software.

Common Types

Type	What it protects against
Antivirus	Traditional viruses attached to programs or files
Anti-malware	Any malicious software, like worms, trojans, or rootkits
Antispyware	Software that spies on your activity or steals data
Anti-trojan	Programs designed to give attackers control of your system
Anti-adware	Software that removes annoying or malicious ads

Modern Endpoint Security

Vendors now often call their products endpoint security suites

These include:

AV/anti-malware scanning

Detection for spyware, trojans, worms, ransomware

Additional protections like firewall or behavior monitoring

Why This Change Happened

Malicious software today is more varied than just viruses

Tools like password crackers, ransomware, and spyware are common threats

AV alone is no longer enough; endpoint security handles all types of malware

Endpoint Firewalls (Easy Explanation)

Firewalls are security tools that control and filter network traffic.

They act like security guards at the doors of your network and devices.

1 Network Firewalls

Placed at the boundary of a network

Protects all systems behind it

Controls:

What comes into the network

What goes out of the network

Think of it as:

A big security gate for the office building

2- Host-Based (Endpoint) Firewalls

Installed directly on a device (laptop, desktop, server)

Protects the individual system

Critical for:

Laptops connecting to public Wi-Fi

Systems outside the office network

Think of it as:

A personal security guard for each device

3- Why Both Are Needed

Network firewall protects the office network

Host-based firewall protects devices when they are outside the network

This is part of defense-in-depth

Multiple layers of security → stronger protection

4- Benefits

Prevents attacks from both inside and outside the network

Allows customized protection per device

Helps control both inbound and outbound traffic

5- Limitation

Host-based firewalls can be hard to manage on hundreds or thousands of devices

What is an Endpoint Firewall?

An endpoint firewall (also called a personal firewall) is a firewall that runs on your own computer (PC or laptop), not on a big network device.

Its job is to:

Protect your computer from unwanted network traffic.

It treats your computer as a trusted zone and checks everything coming from the internet into your PC.

1. Packet Filter (Stateful)

This is the basic type.

It looks at:

IP address

Port number

Connection state (is this part of an existing connection?)

In simple words:

It decides: “Is this network packet allowed or not?”

Example:

Allow: Web traffic (port 80, 443)

Block: Unknown ports

2. Permit Only Authorized Services

This means:

Only known and approved services are allowed.

Example:

Allow: Browser, email

Block: Anything else

So even if something tries to connect, it will be blocked unless approved.

3. Block Unauthorized IP Addresses

You can block specific IPs.

Example:

Block: 203.0.113.5 (a hacker's IP)

Allow: Your company server IP

Simple rule:

If the IP is not trusted → block it.

4. Application Control Firewall (Most Common)

This is very powerful and popular.

It controls:

Which programs can use the internet.

Example:

Chrome → Allowed

Unknown.exe → Ask user

So when a new program tries to connect:

The firewall pops up:
"Do you want to allow this app to access the internet?"

This helps stop:

Malware

Spyware

Trojans

5. Operating System Control (Most Strict)

This is the strongest model.

Here:

A program cannot even RUN unless approved.

Not just internet access — even execution is controlled.

Example:

You download a tool → It won't start until approved.

Used in:

High-security systems

Corporate lockdown PCs

Endpoint Security Suites

These are full security packages like:

Antivirus

Firewall

Application control

System lockdown

Their goal:

Desktop lockdown – nothing runs or connects unless trusted.

Very Short Summary Table

Type	What it does (Simple)
Packet Filter	Allows/blocks network packets
Authorized Services	Only approved services work
Block IPs	Blocks bad IP addresses
Application Control	Controls which apps use internet
OS Control	Controls which apps can even run

File Integrity Checking is a security method used to detect if a file has been changed on a system.

How it works in short:

You choose important files to monitor (documents, system files, website files).

The system creates a hash (a unique fingerprint) for each file.

It stores these hashes as a baseline.

Later, it recalculates the hashes.

If a hash is different → the file was modified → an alert is generated.

Steps for Performing File Integrity Checking — Easy Summary

File Integrity Checking (FIC) works by using hashes (digital fingerprints) to detect any file changes.

Here are the steps in simple form:

Choose important files
Select critical files that should not normally change.

Create hashes
Generate a cryptographic hash for each file.

Store hashes securely
Save them in a protected, read-only location.

Recheck regularly
At fixed intervals, recalculate hashes.

Compare results
New hash vs original hash.

Alert if different
If they don't match → file was modified.

Optional: detect new files
Alert if new files appear in monitored folders.

Inclusive Analysis (Watch List)

This method uses a list of keywords to look for.

If a log entry contains one of these words → alert is raised.

Example keywords:

"failed"

"unauthorized"

"error"

Simple idea:

“Alert me only when I see something specific.”

Weakness:
If something bad happens but doesn’t include your keywords → no alert.

2. Exclusive Analysis (Ignore List)

This method uses a list of normal events to ignore.

Everything else → alert is raised.

Simple idea:

“Ignore normal stuff, alert on anything new or unusual.”

This is great when logs are:

Repetitive

Predictable

One-line comparison

Method	How it works
Inclusive	Alert when a keyword is found
Exclusive	Alert when something NOT in the ignore list appears

Application Control is a security method that only allows approved programs to run on a computer.

How it works simply:

A list of trusted applications is created.

Each application is verified using cryptographic hashes.

When a program tries to run:

If it’s on the approved list → it runs.

If it’s not approved → it is blocked.

If the file was modified → it is blocked.

Key Points in Simple Words
1. Only approved apps can run

Unknown or unauthorized programs cannot execute.

2. Hash verification

Even approved apps must be unchanged.
If malware modifies them → they fail the hash check.

3. Two modes

Active mode: Blocks unapproved apps.

Passive mode: Only alerts (does not block).

4. Needs strong management

Works best in environments where:

Software rarely changes.

Updates are controlled and planned.

HIDS (Host-Based Intrusion Detection System) — Easy Summary

HIDS is a security system that runs on individual computers (hosts) to detect suspicious activity.

Instead of watching the whole network like NIDS, it watches:

What is happening inside each computer.

What HIDS Does (Simply)
1. Works on each host

Installed on PCs or servers, not on network devices.

2. More detailed than NIDS

It can see:

File changes

Logs

Running processes

Local network activity

So it gives more precise detection.

3. Detection methods

HIDS uses:

Signature analysis → known attack patterns

Anomaly analysis → unusual behavior

File integrity checking

Log monitoring

Network monitoring

4. Central reporting

Even though detection happens locally:

Alerts and data are usually sent to a central server (SOC) for analysis.

HIDS (Host-based Intrusion Detection System) network monitoring focuses on observing all network traffic to and from a single host. It typically listens on all available interfaces (Ethernet, wireless, and VPN) and uses signature analysis to detect suspicious or malicious activity.

Unlike traditional NIDS, which monitors traffic at specific network points, HIDS acts as a distributed NIDS, with monitoring performed on each individual workstation. This makes it especially powerful for detecting not only inbound attacks but also outbound threats, such as pivoting, internal reconnaissance, lateral movement, and command-and-control (C2) communication.

Overall, HIDS network monitoring provides deep visibility at the host level, enabling detection of backdoor access and internal threats that might bypass centralized network sensors.

HIDS provides deeper and more detailed security monitoring than NIDS because it works directly on each host. It can analyze encrypted traffic after it is decrypted, giving visibility that NIDS cannot achieve. HIDS also offers better accuracy by using detailed host information, which helps reduce false positives.

In addition, HIDS is effective at detecting internal attacks, such as those coming from trusted users, wireless networks, or VPN connections. Since it monitors activity beyond the network perimeter, it serves as the last line of defense, meaning if HIDS detects an attack, all other security controls have likely failed to stop it

HIDS is powerful but has several challenges. It requires frequent updates that are hard to manage on many systems. Each HIDS has limited visibility, seeing only its own host, so it cannot easily detect large or coordinated attacks without a central management system.

At large scale, HIDS becomes expensive due to software, storage, and maintenance costs. It also uses system resources, which can slow down computers. Finally, HIDS generates huge amounts of data, making it difficult for organizations to store and analyze everything effectively.

HIDS is evolving and becoming more advanced. It now monitors applications and websites, not just files, which helps reduce false alarms and detect attacks like SQL injection. It also supports network devices such as routers, switches, and firewalls by tracking configuration and software changes.

In addition, HIDS is no longer a standalone tool; it is now integrated into SIEM systems as a data source. Finally, modern HIDS is starting to prevent attacks, not just detect them, which means it is gradually transforming into HIPS.

HIPS (Host-based Intrusion Prevention System) provides active protection by detecting and stopping both known and unknown attacks. It works by intercepting dangerous system calls, allowing it to block malicious actions before they reach the operating system.

HIPS uses a combination of file integrity monitoring, network monitoring, and application behavior analysis to understand how programs normally work. By correlating all this activity, HIPS can identify suspicious behavior and automatically block attacks when they exceed a defined risk threshold.

HIPS is powerful but has some limitations. It can produce false positives, is complex to implement and maintain, and supports only popular applications, leaving custom software less protected. HIPS does not replace patching or antivirus and uses extra CPU and memory, which can slow systems. Finally, managing a large HIPS deployment requires central consoles and extra staff, adding cost and complexity. Application Behavior Monitoring in HIPS watches how programs normally behave and detects anything unusual. It records the intended actions of an application, monitors it in real time for suspicious activity (like unusual system calls or processes), and can automatically block malicious behavior to prevent attacks.